Fachhochschul-Bachelorstudiengang
SOFTWARE ENGINEERING
A-4232 Hagenberg, Austria

Creating a Visual Studio Extension to Support the Development of Discord Bots with Discord.Net

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Stefan Christian Merwa

Begutachtet von FH-Prof. DI Johann Heinzelreiter

Hagenberg, Februar 2024

software engineering
IKM Fakultät Hagenberg

Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere.

This printed thesis is identical with the electronic version submitted.

Date

28.02.2024

Signature

Acknowledgement

First and foremost, I would like to express my sincere gratitude to my partner, Sabrina Leibetseder, who has always supported and motivated me during the completion of my Bachelor's thesis. I would also like to thank my parents, who have supported me not only emotionally but also financially during my studies. Finally, I would like to thank FH-Prof. DI Johann Heinzelreiter for his extremely helpful tips and constructive feedback.

1 Abstract

This bachelor's thesis introduces a Visual Studio Extension that is about making the creation of Discord bots a lot more comfortable with Discord.NET. The compelling part is how it integrates Microsoft Roslyn and the powerful tools within Visual Studio to give developers support with intelligent code completion through precise code analysis, and a direct line to Discord's API. This project fills a real need for better tools in bot making, aiming to make the whole process more efficient and user-friendly in Visual Studio.

2 Kurzfassung

In dieser Bachelorarbeit wird eine Visual-Studio-Extension vorgestellt, die die Erstellung von Discord-Bots mit Discord.NET vereinfachen soll. Das Besondere daran ist die Verwendung Microsoft Roslyn und den Tools von Visual Studio. Dadurch werden Entwickler mit intelligenter Code-Vervollständigung durch präzise Code-Analyse und einer direkten Verbindung zur API von Discord zu unterstützen. Dieses Projekt deckt einen echten Bedarf an besseren Tools für die Bot-Erstellung ab und zielt darauf ab, den gesamten Prozess in Visual Studio effizienter und benutzerfreundlicher zu gestalten.

Contents

List of Figures

Listings

3 Introduction

In recent years, Discord has emerged as a popular platform not just for communication but for community engagement through custom bots. This thesis explores the simplification of the bot creation process using a Visual Studio Extension and Discord.NET. Discord.NET serves as a .NET library facilitating bot development, while Visual Studio is chosen for its comprehensive support and robust features.

The challenge of developing Discord bots lies in the complexity of Discord's API and the .NET framework, making the process daunting. This extension aims to bridge this gap with an intuitive interface and integrated development tools that streamline bot development. Features such as intelligent code suggestions and automatic error detection enhance productivity and reduce the development timeline.

Furthermore, this thesis utilizes Microsoft Roslyn, a compiler platform for real-time code analysis, to provide feedback on code quality and automate routine coding tasks. This introduction sets the stage for a detailed examination of the extension's development, including its architectural design, functionality, and the integration of underlying technologies.

4 Discord

Discord, a by now popular voice, video, and text chat app, has significantly influenced the way people around the world communicate, especially within various communities and friend groups. Originally developed as a tool for gamers, it has moved on from its initial purpose to more a versatile platform for different conversations, ranging from casual chats to more structured discussions (Discord, 2022).

Origin

The application was founded by Jason Citron and Stanislav Vishnevskiy, who shared a passion for gaming and a vision for improved online communication. Citron, an American entrepreneur, founded OpenFeint, an early mobile gaming platform and Vishnevskiy, a Russian software engineer, had a background in projects like Google Hangouts and Yahoo! Messenger. Their combined expertise in software engineering and game design facilitated the creation of Discord, launched in May 2015 (TheEnlightenedMindset, 2023).

Architecture and Features

Discord is well-known for its server-based structure, which is the core of its functionality. These servers, customizable as public or private consist of voice and text channels. This structure enables many different communities, from study groups and book clubs to remote workers, to establish their purpose within the platform. Another significant aspect of Discord is its comprehensive moderation capabilities. Server administrators have the authority to manage user permissions, establish automated moderation protocols, and utilize features like word filtering (whitelist or blacklist) to maintain a safe and respectful environment for community interactions. Furthermore, Discord's ability to integrate smoothly with other well known platforms contributes to its growing popularity and effectiveness. This integration capability allows users to perform a variety of actions, such as streaming live gaming sessions on Twitch, sharing YouTube videos directly in chat channels, or collaborating on documents via Google Docs. These integrations play an important, if not the most important role in enhancing the overall utility and user experience of Discord (GameInfoHub, 2023).

4.1 Discord Bots

Bots are a key feature of Discord and a type of software that automates tasks and interacts with users in Discord. They are integrated into servers (4) and perform a variety of functions based on the commands and services like moderation, games, music, internet searches, and payment processing based on the programming (Agnihotri, Gautam, and Singh, 2022). Bots use Discord's Application Programming Interface to interact with the server and its users. This API allows bots to send and receive messages, manage

user roles, and more. It is important to note, that one bot can be used for as many servers as required.

Preparation

Discord bot development is a multifaceted process involving understanding the Discord platform, using its APIs, managing bot accounts, and writing and troubleshooting code. Developers must consider aspects like bot complexity, security, permissions, and community engagement. Utilizing available resources, best practices, and community support is key to creating successful and effective Discord bots (ToptalDevelopers, 2022). Before developing the bot, it first needs to be created through the Discord Developer Portal. There is an option to create a new application, which then will contain the bot user that can be configured with various settings like permissions and also provides a token for establishing a connection to Discord's API.

Development

Developing a Discord bot involves choosing a programming language and corresponding library that interfaces with the Discord API. Each language offers unique libraries or frameworks, designed to simplify the bot development process by abstracting the Discord API's complexities. To name a few, amongst the most popular ones are *Discord.js* for Javascript, *discord.py* for Python, *JDA* for Java, *DiscordGo* for Go and *Discord.Net* for C# . The choice of language and library depends on the developer's familiarity with the language, the specific requirements of the bot, and the desired level of abstraction from the Discord API. Each language and library has its advantages and is suited to different types of projects and developer preferences.

4.2 Discord.Net

Discord.Net is an asynchronous, multi-platform .NET Library designed to interface with the Discord API. It is an unofficial .NET API wrapper for the Discord client and also very feature rich by supporting slash commands, interaction frameworks and message components. To get startet with Discord.Net, a good understanding of advanced .NET programming concepts like the Task-based Asynchronous Pattern is required, since the library extensively uses them. There are a lot of tutorials and also a comprehensive guide that provides detailed instructions and examples to help developers (Discord.Net, 2015). There exists a public Discord server for Discord.Net with more than 3000 current members, some of whom are willing to offer support and answer questions about the library. For common C# questions there also exists a Discord server with almost 50.000 members, where there is a channel dedicated to Discord development.

5 Foundation

To achieve the wanted capabilities for the Visual Studio Extension, multiple technologies are required. First a core understanding of how Visual Studio Extensions are developed and what features and possibilities they bring to the table is necessary. To effectively implement this process, it's essential to first set up a way to retrieve data from the Discord API. Since it's common to develop just one bot per C# project, it makes sense to trigger the process for each individual project. The way of implementation will be explained in more detail later. This data is then used for code completion utilizing the Editor API. Additionally, using the said API requires working with the Managed Extensibility Framework (MEF), since the Editor API depends on it. An important step in this process is figuring out exactly when and what type of data should be made available for code completion, and this is where MS Roslyn comes in, offering precise code analysis to make these determinations.

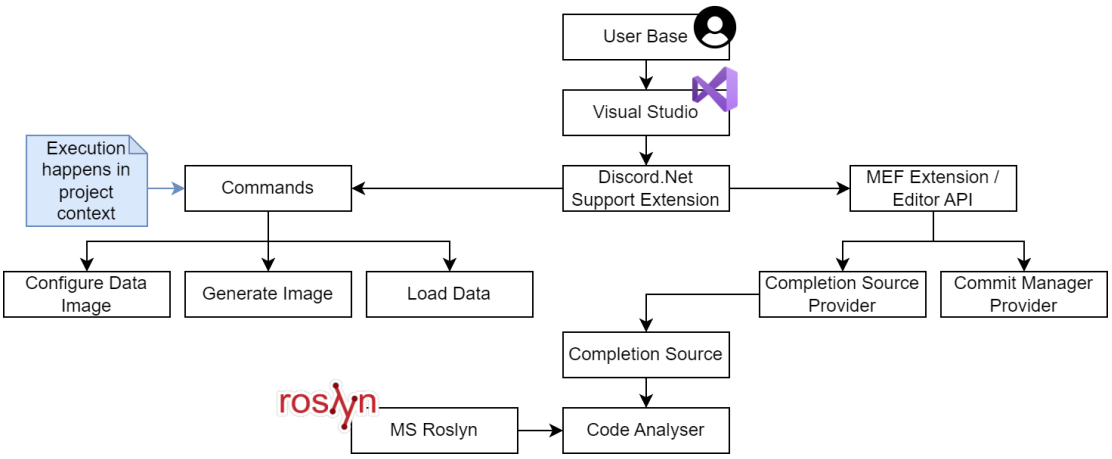

Figure 1: Workflow overview to acquire the wanted solution

To clarify Figure 1 in a more detailed manner, imagine a user who is coding in Visual Studio and has installed an extension that provides the necessary features mentioned in the Introduction (Section 3). This extension includes various Commands that can be accessed by right clicking a project. For interacting with the Editor API, a separate project is created and then added to the main project as a MEF component. For reference, it would also be possible to create the MEF components in the main project, but in terms of readability and maintainability it makes sense to put them in a seperate project. And that is where the user's code is analyzed using MS Roslyn. The details of these steps are thoroughly explained in the following sections.

5.1 Visual Studio Extension Development

To participate in Visual Studio Extensibility (VSIX), the workload needs to be installed.

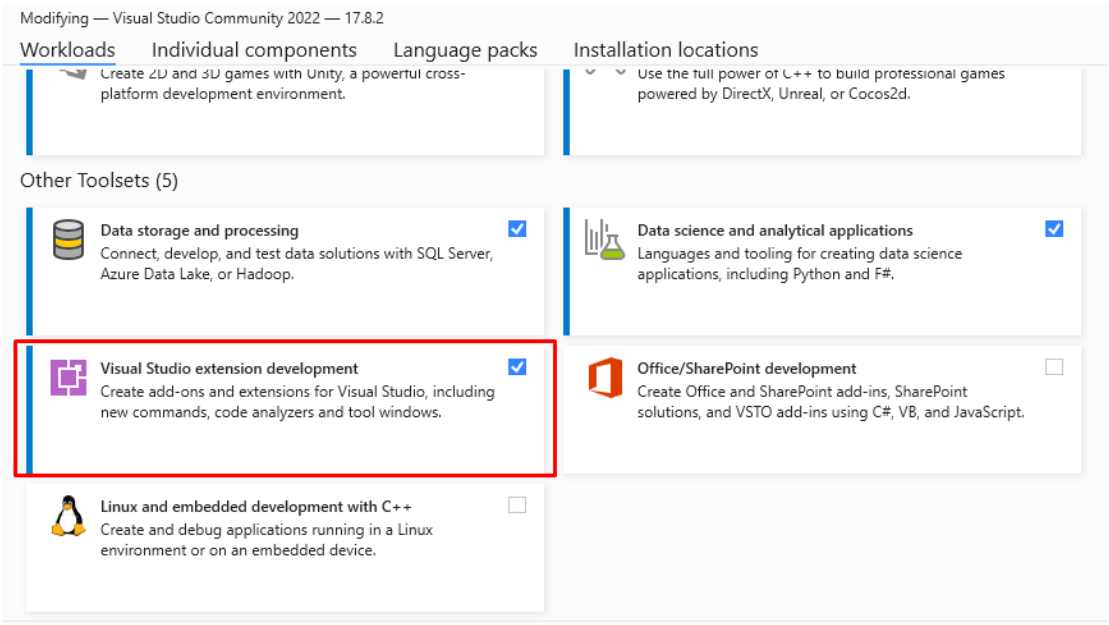

Figure 2: Visual Studio Installer

The types of extensions are quite varied. They can range from supporting new languages not included in Visual Studio, to productivity tools that enhance the core IDE experience, to domain-specific designers for specific scenarios like data design or cloud support. In Visual Studio, many different parts can be extended with the most common features to be commands, menus, toolbars, windows, IntelliSense, and projects (Microsoft, 2023). For example, there exists the Visual Studio Marketplace, where many Extensions are available for free or are even open sourced.

5.1.1 Anatomy

File Structure

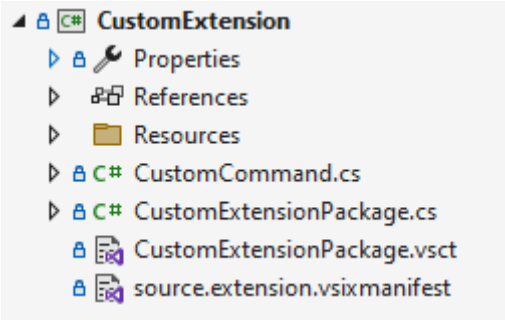

Figure 3: File structure of a VSIX Project

Figure 3 shows the file structure of an extension project. The manifest file (*source.extension.vsixmanifest*) is the primary and an XML based file that provides essential information about the extension to Visual Studio. It registers all the components of the extension and is the only required file in a VSIX project.

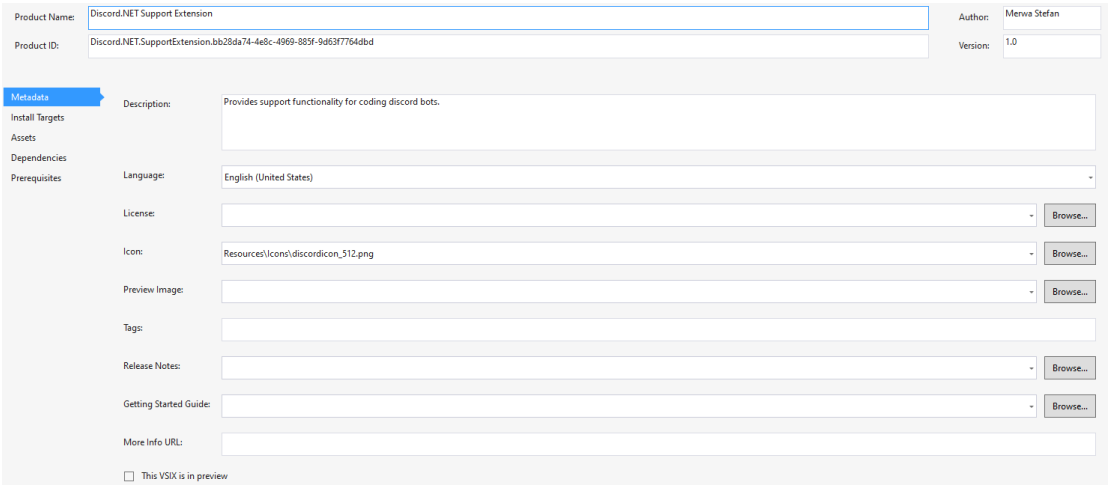

Figure 4: Design view of the .vsixmanifest file

Figure 4 shows the contents of the Manifest for a Visual Studio Extension. This context menu pops up after double clicking on the file, so editing is made easier.

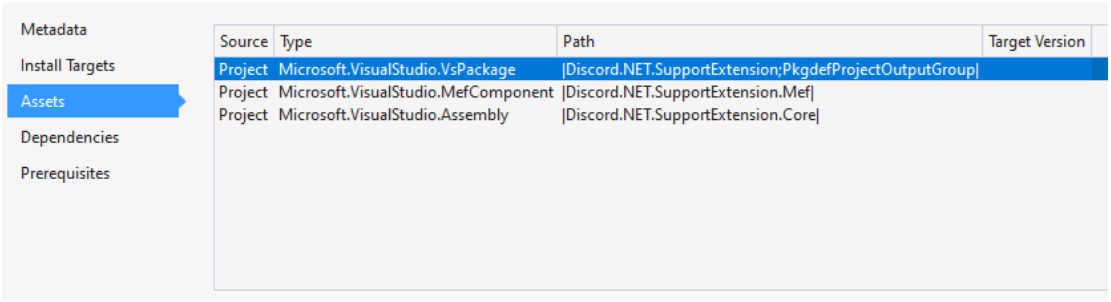

Figure 5: .vsixmanifest file assets

As mentioned in Section 5 the interaction with the Editor API requires the usage of the MEF. That leads to the creation of a second project that will be imported to the main project as a MEF component. So a new asset is required, that goes with the source, type and path, as in this case the source is a *project*, the type is a *MefComponent* and the Path refers to the second project *Discord.NET.SupportExtension.Mef* which is illustrated in Figure 5. (Microsoft, 2021c)

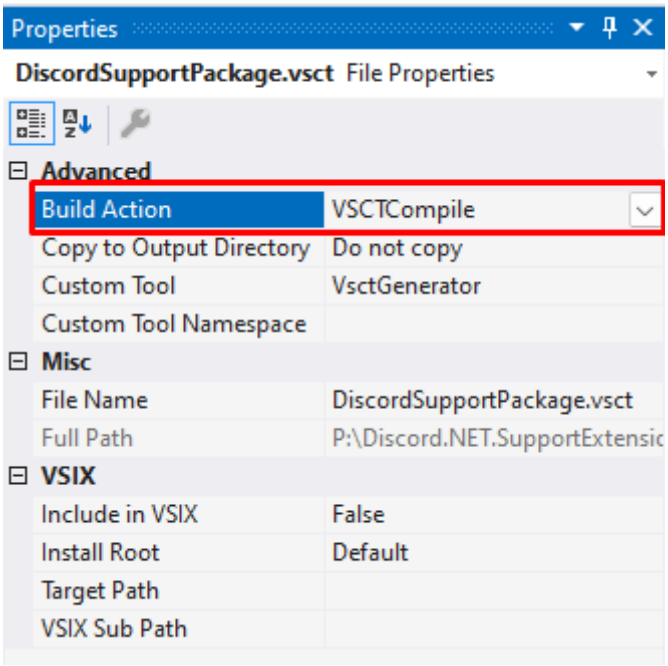

Figure 6: Properties of a VSIX Command Table file

Commands within the extension are defined in the command table (*DiscordSupport-Package.vsct* in Figure 3), another XML file. This file includes definitions for button commands, menus, and keyboard shortcuts. Figure 6 shows that its contents are compiled as an Assembly Resource, which Visual Studio utilizes to build its command table menu structure. However, this file only declares components and does not manage command executions. The Package.cs file (*DiscordSupportPackage.cs* in Figure 3), serves as the primary entry point for most extensions. This file typically contains command han-

dlers, tool windows, options pages, services, and other components. (Microsoft, 2021c)

Compilation

The project is compiled into a *.vsix* file, which can be found in either the */bin/debug* or */bin/release* folder, depending on the build configuration set for the current solution. The workload for Visual Studio extension development includes specialized MSBuild targets and tasks specifically designed for managing VSIX projects. During the build process of the VSIX project, it is automatically deployed to the Experimental Instance of Visual Studio. (Microsoft, 2021c)

Experimental Instance

To protect the main Visual Studio development environment from potential alterations by untested applications, the Visual Studio SDK offers an experimental space for testing and experimentation. This enables the possibility to develop new applications using the regular Visual Studio interface, but they are run in a separate, isolated Experimental Instance. This ensures that the main development environment remains unaffected. Furthermore, every application equipped with a VSIX package automatically launches the Visual Studio Experimental Instance in debug mode when the application is run.

5.1.2 Commands

A Visual Studio Command provides functionality that can be executed within the IDE. They can range from simple tasks like adding a new file to a project, to more complex tasks like executing test collections. Commands can be triggered in various ways like through menu items, toolbar buttons, keyboard shortcuts or even programmatically. They are meant to impact the general Visual Studio experience by boosting efficiency through automating manual operations.

Definition

Every button in every menu is a command. To add commands to an extension, they need to be defined in the command table.

Listing 1: Custom command group definition

```
<Symbols>
  <!-- This is the package guid. -->
  <GuidSymbol name="guidCustomExtensionPackage" value="{ad9143c7
      -5915-47e5-a9ac-d46c387072db}" />

  <GuidSymbol name="CustomGroup" value="{119bf355-ef7f-4cdf-af9a-8
      fe6b13c8658}">
    <IDSymbol name="CSharpProjectMenuGroup" value="0x100" />
  </GuidSymbol>

  <GuidSymbol name="CommandSet" value="{c97d316c-21f9-4def-9731-4
      d63790b08fd}">
    <IDSymbol name="CustomCommand" value="0x0100" />
  </GuidSymbol>

  ...
<\Symbols>
```

Listing 1 displays how the symbols are defined for further usage in the command table. These symbols will now be used inside this file with a settable priority tag that determines the ordering of groups or items within the same parent menu or toolbar.

Listing 2: Custom command group definition

```
<Groups>
  <Group guid="CustomGroup" id="CSharpProjectMenuGroup" priority="0
      x003" />
</Groups>
```

Listing 2 shows how to use the Group that was defined in Listing 1. This acts as parent element for other components.

Listing 3: Custom command button definition

```
<Buttons>
  <Button guid="CommandSet" id="CustomCommand" priority="0x0100" type=
      "Button">
    <Parent guid="CustomGroup" id="CSharpProjectMenuGroup" />
    <Icon guid="guidImages" id="bmpPic1" />
    <Strings>
      <ButtonText>Click me</ButtonText>
    </Strings>
  </Button>
</Buttons>
```

In Listing 3 the required buttons are defined with a parent element, an icon and a text

that is displayed on the button. The icon is defined as a Bitmap, which is not mandatory for this showcase.

Listing 4: Command placement

```
<CommandPlacements>
  <!-- Project Context -->
  <CommandPlacement guid="CustomGroup" id="CSharpProjectMenuGroup"
      priority="0x004">
    <Parent guid="guidSHLMainMenu" id="IDM_VS_CTXT_PROJNODE" />
  </CommandPlacement>
</CommandPlacements>
```

Everything inside the *CustomGroup* is placed at the provided parent element, in this case, the project context menu, as shown in 4.

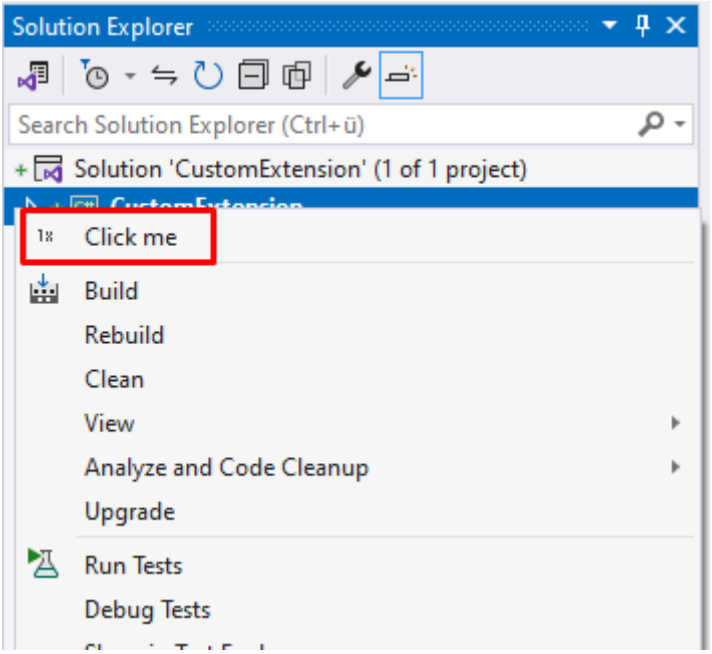

Figure 7: Result of the command definition

After defining the command and coherent elements, the Click-Me button will appear on top of all other buttons when right clicking on a project as shown in Figure 7, considering that the just implemented extension is installed.

Execution

Now that the command is defined in the command table, the invocation logic must be created. Every command is accessed via a GUID and ID from Visual Studio (the ones defined in the command table as displayed in Listing 1). Keep in mind, that if multiple commands are used, creating a base class might be a wise decision.

Listing 5: Possible command base

```
1 public abstract class CommandBase {
2   protected AsyncPackage Package { get; }
3   protected abstract Guid CommandSet { get; }
4   protected abstract int CommandId { get; }
5
6   protected CommandBase(AsyncPackage package, IMenuCommandService
        commandService) {
7     this.Package = package ?? throw new ArgumentNullException(nameof(
          package)); ;
8     if (commandService == null) throw new ArgumentNullException(nameof
          (commandService));
9
10    CommandID menuCommandId = new CommandID(CommandSet, CommandId);
11    OleMenuCommand menuCommand = new OleMenuCommand(this.Execute,
          menuCommandId);
12
13    commandService.AddCommand(menuCommand);
14  }
15
16  protected abstract void Execute(object sender, EventArgs e);
17  protected IAsyncServiceProvider AsyncServiceProvider => Package;
18  protected IServiceProvider ServiceProvider => Package;
19 }
```

Listing 5 shows a possible base class for commands in Visual Studio. It covers the necessary values that are required by a command that include the *CommandSet* and *CommandId* which are required for the command to be mapped correctly. Code line 11 shows that the *Execute* Method is passed as event handler to the menu command, so it cannot be async without loosing the execution state. There exists a well known workaround, for which it also makes sense to create a base class.

Listing 6: Base class for async command handling

```
1 public abstract class AsyncCommandBase : CommandBase {
2   private readonly Action<Exception> onException;
3   private bool isRunning = false;
4
5   protected AsyncCommandBase(AsyncPackage package, IMenuCommandService
        commandService, Action<Exception> onException) : base(package,
      commandService) {
6     this.onException = onException;
7   }
8
9   protected override async void Execute(object sender, EventArgs e) {
10    try {
11      if (isRunning)
12        throw new CommandException($"Command {this.GetType().Name} is
              already running.");
13
14      isRunning = true;
15      await ExecuteAsync(sender, e);
```

```
16      }
17      catch (Exception ex) {
18        onException?.Invoke(ex);
19      }
20
21      isRunning = false;
22    }
23
24    protected abstract Task ExecuteAsync(object sender, EventArgs e);
25  }
```

Listing 6 shows said workaround. Asynchronous methods with void as return type are a very bad practice, because the execution results in *Fire and Forget*. This workaround provides a Delegate that will be invoked if an exception occurs while command execution.

Listing 7: Custom command

```
1  internal sealed class CustomCommand : AsyncCommandBase {
2    protected override Guid CommandSet => PackageGuids.CommandSet;
3    protected override int CommandId => PackageIds.CustomCommand;
4
5    internal CustomCommand(AsyncPackage package, IMenuCommandService
         commandService, Action<Exception> onException) : base(package,
         commandService, onException) {
6      // Initialization work
7    }
8
9    public static CustomCommand Instance { get; private set; }
10   public static async Task InitializeAsync(AsyncPackage package,
         IMenuCommandService commandService, Action<Exception>
         onException) {
11     Instance = new CustomCommand(package, commandService, onException)
           ;
12   }
13
14   protected override async Task ExecuteAsync(object sender, EventArgs
         e) {
15     // Command Execution
16   }
17 }
```

After creating these classes, the final command class can look like Listing 7 displays. The *PackageGuids* class is generated using the command table, so it provides the required GUIDs and IDs for all defined commands. It appears in certain scenarios, that the *PackageGuids.cs* file is not generated, in this case the *CommandSet* and *CommandId* can simply be created inside the command by copying the values from the command table (in this case the values seen in Listing 1).

Listing 8: Calling command's InitializeAsync in package initialization

```
protected override async Task InitializeAsync(CancellationToken
    cancellationToken, IProgress<ServiceProgressData> progress) {
  IMenuCommandService commandService = await this.GetServiceAsync(
      typeof(IMenuCommandService)) as IMenuCommandService;
  await CustomCommand.InitializeAsync(this, commandService,
      OnException);
}
```

It is important to call the Initialization method *InitializeAsync* from inside the package class (*DiscordSupportPackage.cs* from Figure 3) to create the Singleton instance of said command like Listing 8 shows. (Microsoft, 2021c)

5.1.3 Displaying Custom Windows

When working with custom commands, the display of a custom implemented window with i.e. WPF might be required. The *IVsUIShell* provides said functionality.

Listing 9: Displaying a custom window

```
public static void Show(System.Windows.Window window) {
  // Is required since the Visual Studio Shell should only be used on
      the main thread
  ThreadHelper.ThrowIfNotOnUIThread();

  IVsUIShell uiShell = (IVsUIShell)Package.GetGlobalService(typeof(
      SVsUIShell));
  Assumes.Present(uiShell);

  uiShell.GetDialogOwnerHwnd(out IntPtr hwnd);
  WindowHelper.ShowModal(window, hwnd);
}
```

Listing 10 shows how to access the *IVsUIShell*, which always needs to happen on the main thread, and opening a new custom window using the *WindowHelper* class.

5.1.4 Logging

Besides opening custom windows, it is important to keep the user of the extension updated about what is happening in the background. Especially when operations take a long time. This can happen via creating log files, but a much more suited approach is to create a new *IVsOutputWindowPane* that will then receive the log statements.

Listing 10: Displaying a custom window

```
1  private static IVsOutputWindowPane pane;
2  public static void InitOutputLog(string paneName) {
3    ThreadHelper.ThrowIfNotOnUIThread();
4
5    IVsOutputWindow logWindow = AsyncPackage.GetGlobalService(typeof(
         SVsOutputWindow)) as IVsOutputWindow;
6    Guid guid = Guid.NewGuid();
7    logWindow.CreatePane(ref guid, paneName, 1, 1);
8    logWindow.GetPane(ref guid, out pane);
9  }
10
11 public static void OutputWindowFunc(LogStatement obj) {
12   Func<Task> logTask = new Func<Task>(async () => {
13     await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();
14     pane.OutputStringThreadSafe(obj.ToString() + "\n");
15   });
16
17   ThreadHelper.JoinableTaskFactory.Run(logTask);
18 }
```

Listing 10 shows how the pane is created using the *IVsOutputWindow* and can then be accessed by loggers to provide logs to the created output window pane. It is also important to not invoke tasks by hand, but let the *ThreadHelper.JoinableTaskFactory* handle task execution to avoid deadlocks.

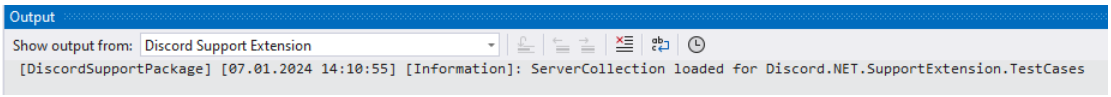

Figure 8: Custom output window pane

For instance Figure 8 shows that there exists a new *IVsOutputWindowPane* called "Discord Support Extension" that provides context specific information.

5.2 Roslyn

Microsoft's Roslyn, also known as the .NET Compiler Platform SDK, represents a significant change in how .NET handles compilation and provides extensive APIs for various compilation phases. Roslyn is essentially a set of services built around two key principles: Extensibility and Maintainability. This design allows for layers above and integration with third party plugins, offering well documented and supported public APIs. External developers can leverage these services to do a variety of innovative things. Some of these can be crafting their own tools tailored to any specific layer of programming tasks or developing simple plugins like diagnostic analyzers, code fixes, refactorings, and IntelliSense providers for certain layers (Vasani, 2017). Roslyn plays an essential role to achieve the wanted goal with the Visual Studio Extension, because it enables the analysis of a users code during the coding process. This is required because the completions pushed to the Editor API are context dependent, so it is important that the context is determined live while the user is coding.

5.2.1 Compiler phases

Roslyn deconstructs the compiler into distinct phases and exposes managed APIs for each phase, making language compilers a service (Developer.com, 2012). The .NET Compiler Platform SDK is a set of tools that simplifies building C# and VB compilers, following steps from analyzing text to creating executable code. It provides tools for each step to work with the code being compiled (Microsoft, 2021b).

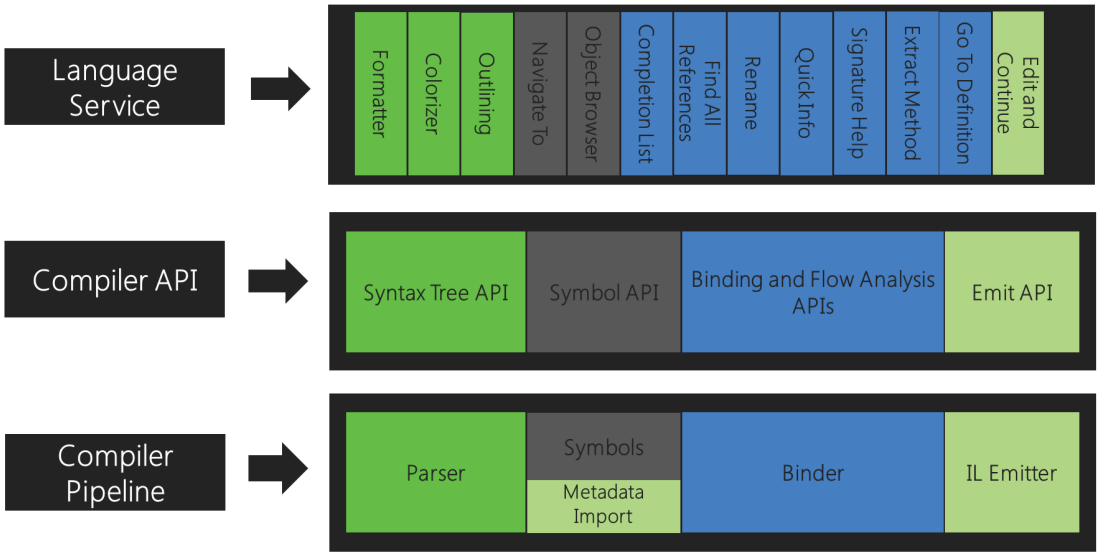

Figure 9: Compiler pipeline functional areas[1]

Figure 9 indicates that upon the Compiler Pipeline the Compiler API and Language Service are built. To analyse ones code, the Compiler API is required to access syntax trees and the Symbol API which contains the semantic information.

[1]Source: `https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/compiler-api-model/compiler-pipeline-lang-svc.png` (2024-01-07)

5.2.2 Code Analysis

Code analysis in general is a big topic, but in this scenario, where the users code is analysed after the IntelliSense process is triggered, there are only a few key concepts required. These concepts consist of Workspaces, Syntax and Semantics, so a core understanding of how these things work is required.

Workspaces and Solutions

Workspaces are essential for code analysis and refactoring across entire solutions. It organizes project information into a unified object model, allowing easy access to important compiler elements like syntax trees and semantic models, without file parsing or dependency management. Within an IDE, workspaces reflect real-time changes, updating the model as users proceed to make updates like writing code or changing properties (Microsoft, 2021d). Even though workspaces can change with each key press, the solution model can be worked with independently. Solutions are immutable models containing projects and documents, allowing for sharing without conflicts or duplication. Once a solution instance is obtained from the Workspace.CurrentSolution property, it remains unaltered. Modifications to solutions involve creating new instances based on existing ones and applying these changes back to the workspace. Projects within the solution model contain source code documents, parsing and compilation settings, and references, facilitating direct access to compilations. Documents in the model represent individual source files, from which text, syntax tree, and semantic model can be accessed. (Microsoft, 2021d)

Listing 11: Roslyn Workspace

```
1 IComponentModel componentModel = (IComponentModel)Package.
      GetGlobalService(typeof(SComponentModel));
2 VisualStudioWorkspace workspace = componentModel.GetService<Microsoft.
      VisualStudio.LanguageServices.VisualStudioWorkspace>();
3
4 Solution solution = workspace.CurrentSolution;
5
6 foreach (Project project in solution.Projects) {
7   Console.WriteLine($"Project: {project.Name}");
8
9   foreach (Document document in project.Documents) {
10    Console.WriteLine($"\tDocument: {document.Name}");
11  }
12 }
```

Listing 11 shows how to access Visual Studio's workspace and getting a snapshot of the current solution that is open inside the workspace.

Syntax

Syntax Trees provide a detailed, hierarchical representation of source code's structure,

essential for analyzing and manipulating code in tools like IDEs. They are immutable, ensuring thread safety and full fidelity, meaning they represent every piece of source text, including whitespaces and comments. Syntax Nodes within these trees represent the syntactic constructs like expressions or statements. They are interconnected which means they are forming the tree's structure. Syntax Tokens are the smallest elements, representing keywords, identifiers, and punctuation, playing a key role in the tree's construction and the interpretation of the code's syntax. Syntax Trivia covers more unimportant text like spaces or comments. Spans indicate the text range of nodes or tokens. Syntax Kinds are used to categorize syntax elements, supporting their identification. (Microsoft, 2021f)

Example

Listing 12: Syntax Example

```
1  SyntaxTree tree = CSharpSyntaxTree.ParseText(@"
2  public class CustomClass {
3    public void CustomMethod() {
4    }
5  }");
6
7  CompilationUnitSyntax root = tree.GetRoot() as CompilationUnitSyntax;
8
9  foreach (ClassDeclarationSyntax classDeclaration in root.
       DescendantNodes().OfType<ClassDeclarationSyntax>()) {
10   Console.WriteLine($"Class: {classDeclaration.Identifier.ValueText}")
         ;
11
12   foreach (MethodDeclarationSyntax methodDeclaration in
         classDeclaration.DescendantNodes().OfType<
         MethodDeclarationSyntax>()) {
13     Console.WriteLine($"Method: {methodDeclaration.Identifier.
           ValueText}");
14   }
15 }
```

Listing 12 shows a string representing valid C# syntax that is used to create a *SyntaxTree*. Then the root is accessed to iterate through the descendant nodes to identify classes and methods, which identifier values are printed to the console.

Listing 13: Syntax Example Output

```
1  Class: CustomClass
2  Method: CustomMethod
```

Listing 13 shows the output of the executed Listing 12.

Compilation and Symbols

The Semantic model in compilation extends beyond syntax, interpreting elements like types and variables, essential for understanding code without direct access to its source, such as in libraries. It represents a deep layer of analysis, connecting names in code to their meanings and behaviors through symbols, each signifying a unique element like a class or method, derived from compiler insights or imported metadata. Symbols and the Semantic model provide a comprehensive framework for navigating and interpreting code's logical structure, essential for tasks like type checking and resolving references within a compilation's immutable context. (Microsoft, 2021e)

Example

Listing 14: Semantics Example

```
SyntaxTree tree = CSharpSyntaxTree.ParseText(@"
public class CustomClass {
  public void CustomMethod(int customParameter) {
  }
}");

SyntaxNode root = tree.GetRoot();
Compilation compilation = CSharpCompilation.Create("CustomCompilation")
.AddReferences(MetadataReference.CreateFromFile(typeof(object).Assembly.Location))
.AddSyntaxTrees(tree);

SemanticModel model = compilation.GetSemanticModel(tree);
MethodDeclarationSyntax methodDeclaration = root.DescendantNodes().OfType<MethodDeclarationSyntax>().First();
IMethodSymbol symbol = model.GetDeclaredSymbol(methodDeclaration);

Console.WriteLine($"Method: {symbol.Name}");
foreach (IParameterSymbol parameter in symbol.Parameters) {
  Console.WriteLine($"Parameter: {parameter.Name}, Type: {parameter.Type}");
}
```

Listing 14 shows a string representing valid C# syntax that is used to create a *SyntaxTree* and builds the compilation based on that tree. Then the compilation is used to acquire a *SemanticModel*, that is then used to get the symbol from the first found *MethodDeclarationSyntax*. All parameters are iterated and its name and type are printed to the console.

Listing 15: Semantics Example Output

```
1   Method: CustomMethod
2   Parameter: customParameter, Type: System.Int32
```

Listing 15 shows the output of the executed Listing 14.

5.2.3 Syntax Visualizer in Visual Studio

"The Syntax Visualizer enables inspection of the syntax tree for the C# or Visual Basic code file in the current active editor window inside the Visual Studio IDE. The visualizer can be launched by clicking on View > Other Windows > Syntax Visualizer. You can also use the Quick Launch toolbar in the upper right corner. Type "syntax", and the command to open the Syntax Visualizer should appear" (Microsoft, 2022)

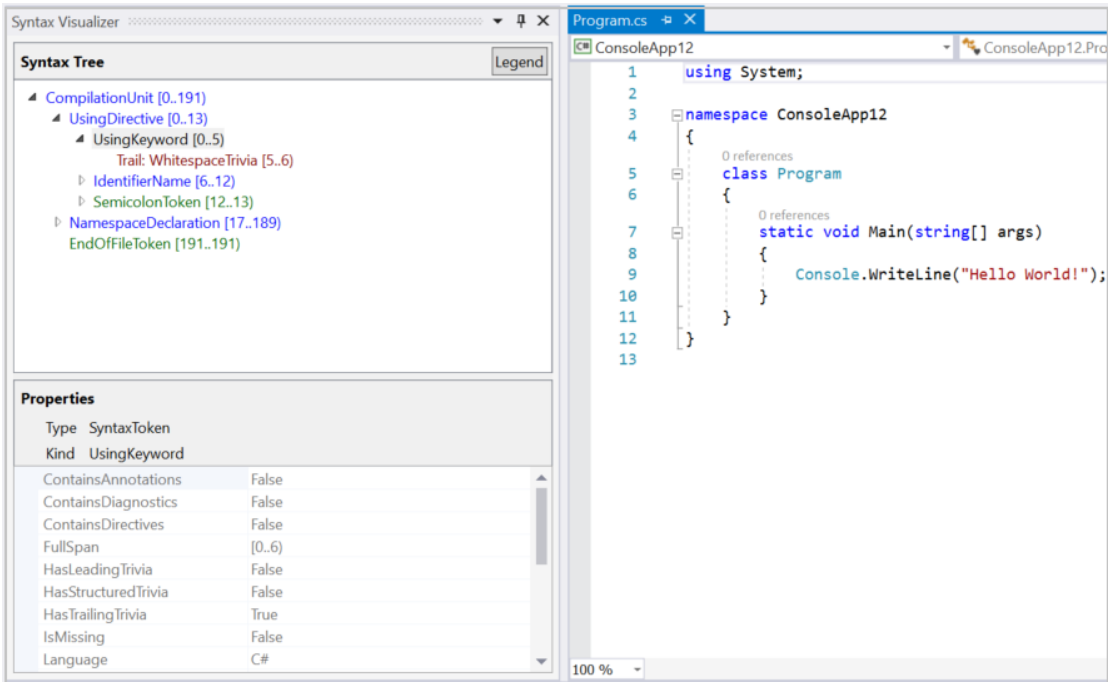

Figure 10: Syntax Visualizer[2]

[2]Source: https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/syntax-visualizer/visualize-csharp.png (2024-01-07)

5.3 Editor API

The Microsoft Visual Studio Editor API contains open source layers of the Visual Studio editor, including public API definitoins and implementations for text model, logic and editor operations. It is designed for extensibility creators to integrate with Visual Studio (Microsoft, 2021a).

5.3.1 Async Completion API

The Async Completion API is a subcomponent of the Editor API focusing on asynchronous operations to enhance IntelliSense features. The most important one for this use case being the addition of completion items using the *IAsyncCompletionSourceProvider* with an *IAsyncCompletionSource* as well as handling how these items will be committed to the text buffer utilizing the *IAsyncCompletionCommitManagerProvider* with an *IAsyncCompletionCommitManager*. Visual Studio uses the Managed Extensibility Framework to discover and instantiate components that inherit from interfaces like *IAsyncCompletionSourceProvider* and *IAsyncCompletionCommitManagerProvider*. MEF scans the assembly metadata for exported components marked with specific attributes indicating their purpose and capabilities. When Visual Studio requires these services, MEF composes the parts together, linking the providers to the editor based on the exported metadata. This mechanism allows for a decoupled, extensible architecture where extensions can be developed and integrated seamlessly into the Visual Studio environment, enhancing the editing and coding experience without direct modification to the core IDE. (Wieczorek, 2018b)

5.4 Managed Extensibility Framework

The Managed Extensibility Framework (MEF) is a tool designed to build extensible and lightweight applications. Since MEF is used by Visual Studio like mentioned in Subsection 5.3.1, a basic understanding of how exporting with MEF works is required. It is basically a dependency injection (DI) container but differs from other containers in its focus on extensibility and part discovery. Unlike traditional DI containers designed mainly for dependency management within an application, MEF aims to make applications extendable by allowing parts to be added or removed without requiring changes to the core application. It uses a catalog and composition model to discover and compose parts at runtime, which makes it ideal for applications that require incorporate extensions or plugins.

Composition container

At the heart of the MEF composition model lies the composition container, serving as the repository for all available parts and leading their composition. Composition is the matching up of imports to exports.

Imports and Exports

Listing 16: ICustomHandler for MEF example

```
1 public interface ICustomHandler {
2   string Handle(string input);
3 }
```

Listing 16 shows the design of an interface that is required for later usage.

Listing 17: CustomHandler export for MEF example

```
1   [Export(typeof(ICustomHandler))]
2   class CustomHandler : ICustomHandler {
3   }
```

Listing 17 shows how the Export attribute is used. For a successful match between an import and an export, both must share the same contract. If an export is associated with a different contract, such as *typeof(CustomHandler)*, it won't match an import looking for a different contract, leading to the import not being fulfilled. The contract must be an exact match.

Listing 18: ICustomHandler import for MEF example

```
1 [Import(typeof(ICustomHandler))]
2 public ICustomHandler customHandler;
```

Listing 18 displays how to import an exported component. In MEF, every import is associated with a contract that determines which exports it matches with. This contract can be an explicit string or automatically generated based on a type, like the *ICustomHandler* interface in this context. Any export that declares a matching contract can fulfill this import. The contract is not necessarily tied to the type of the importing object. Even though the object type might be *ICustomHandler*, the contract can differ. By default, MEF uses the type of the import to determine the contract, unless explicitly specified otherwise.

6 Building the Visual Studio Extension

Now that the necessary information was provided in Section 5 (Foundation), this Section deals with the built Visual Studio Extension through some code examples and explanations. However, as it is not possible to explain the entire extension, only the most important things are mentioned. The fully functional Visual Studio extension and therefore also the complete code is available on Github: `https://github.com/TheHandsomeBanana/Discord.NET.SupportExtension`.

The following Subsections are going to show only the key concepts of how to create a project that is capable of the described features. These include the

1. creation of a new Visual Studio extension project
2. specific descriptions of the required commands. The core implemention of a command is shown in Subsection 6.2. It also shows how to connect to the Discord API using the *Discord.Net* NuGet package.
3. explanations of the MEF components used to interact with the Editor API.
4. explanations of how the analysers built with Roslyn function, without diving into source code.

6.1 Creating a new Visual Studio Extension Project

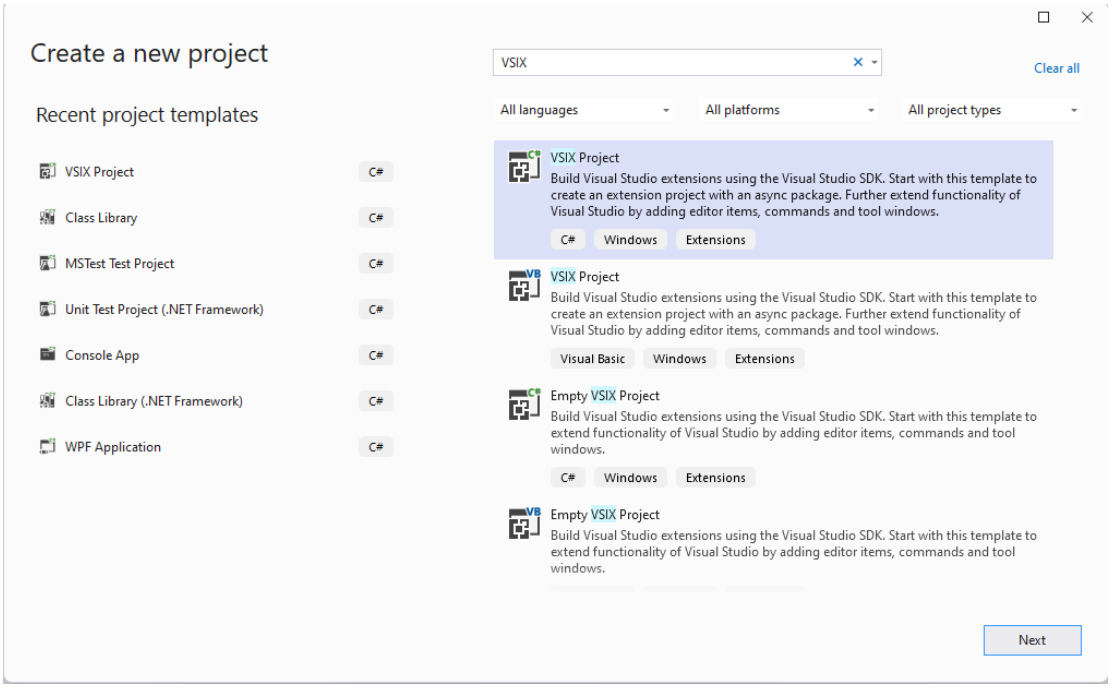

Figure 11: References of the newly created Visual Studio Project

To create a new Visual Studio Extension, it is not required to start from scratch, there already exists a template as Figure 11 shows.

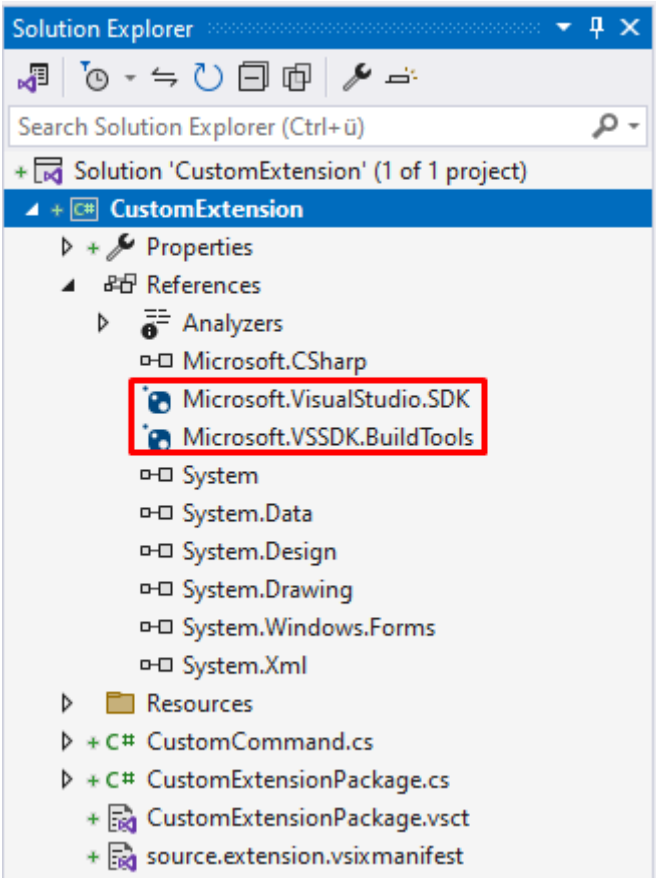

Figure 12: References of the newly created Visual Studio Project

The necessary NuGet packages for extension development are already referenced by default in this template as Figure 12 shows.

6.2 Commands in Project-Context

As already mentioned in Section 5 there are 3 commands required for this use case. These commands build the core of the data generation, that is then furtherly processed for completion through the Editor API.

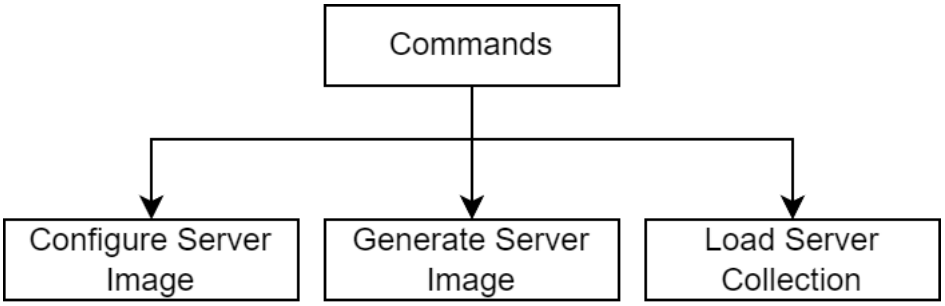

Figure 13: Commands displayed

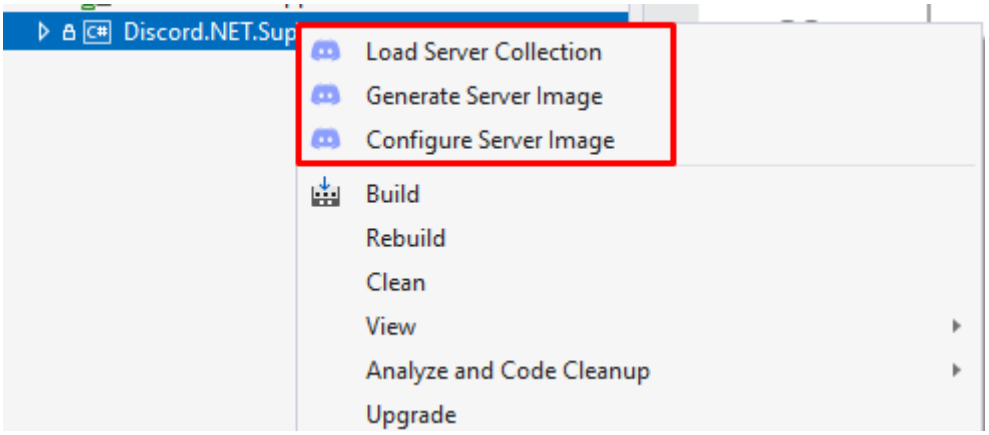

Figure 14: Commands in project context

Figure 14 and 13 show the 3 required commands. The *Configure Server Image* command is used to open a custom implemented window that enables configuration possibilities to the user. *Generate Server Image* will trigger the data generation process, which is going to connect to the Discord API and load the required entities. The *Load Server Collection* command reads the generated data based on the project to make it usable for the completion process.

Configure Server Image

Since the data for the IntelliSense process should be cached to keep performance up to track, a way to get this data from the API by executing this command is required. To acquire said functionality, a custom window is implemented that takes the users bot token and additional settings like encryption methods, which can then be used to download all servers data that this bot is part of.

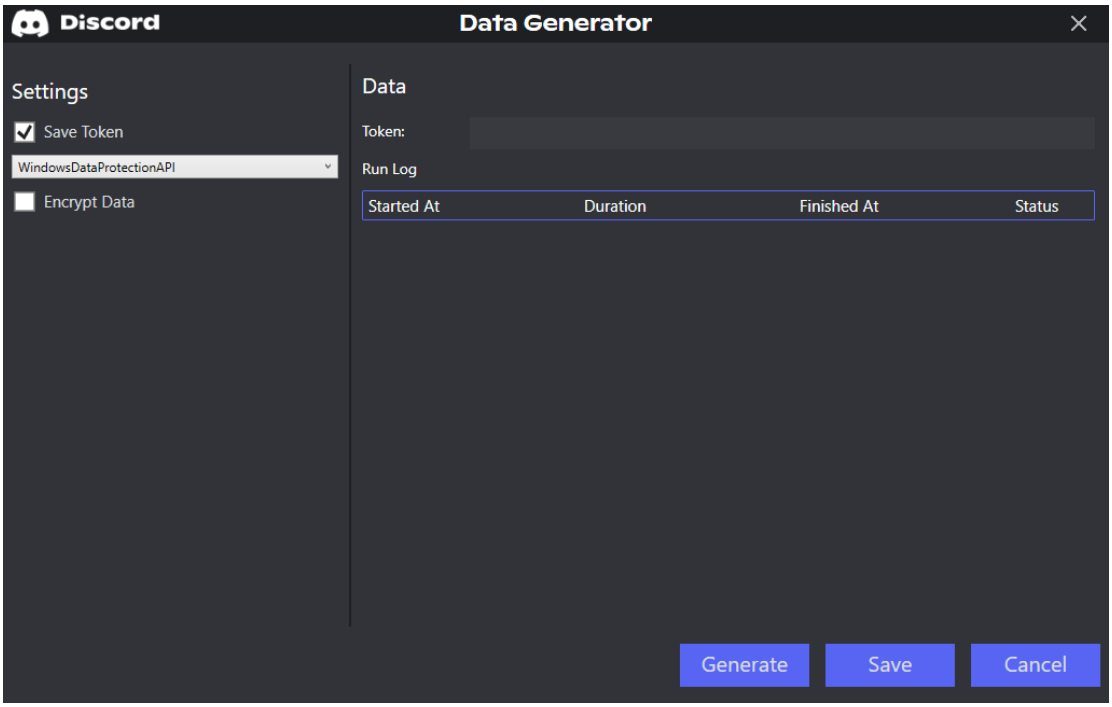

Figure 15: Command that configures how the data will be generated

Figure 15 shows the custom window that is used to configure the data generation process. It is possible to save the token, which will then be stored using the selected cryptography mode. Supported ways to encrypt and decrypt the token are the Windows Data protection API (DPAPI) and AES. Same is possible for the downloaded data. When clicking *Save*, the configurations data is saved as JSON and added to the project as a new file called *DiscordServerImageConfig.json*.

Listing 19: DiscordServerImageConfig.json

```
{
  "SaveToken": true,
  "EncryptData": false,
  "TokenEncryptionMode": null,
  "DataEncryptionMode": null,
  "TokenKeyIdentifier": "",
  "DataKeyIdentifier": null,
  "Token": "",
  "RunLog": []
}
```

Listing 19 shows how the configuration looks like in JSON format. When extending this whole process, it definitely makes sense to use a database instead of storing information like a run-log inside the configuration file.

Generate Server Image

To generate the server image, the bot token is used to connect to Discord's API and load the required data.

Listing 20: Load data from the Discord API

```
// This service is used to first login with the provided token and
    then download the data from the Discord API
IDiscordEntityService entityService = container.Resolve<
    IDiscordEntityService>(nameof(DiscordRestEntityService));
try {
  ...
  // Connect to the Discord API
  await entityService.Connect(token);
  if (entityService.Ready) {
    DiscordServerCollection serverCollection = await entityService.
        LoadEntities();

    // The loaded data will be saved to a directory inside appdata
    await entityService.SaveToFile(currentCachePath, serverCollection)
        ;

    ...
  }
  await entityService.Disconnect();
}
finally {
  await entityService.DisposeAsync();
}
```

Listing 20 represents a snippet of the Generate Server Image command, where the *IDiscordEntityService* is used to download the required data using the provided bot token. This service is a custom implementation utilizing Discord.Net's functionality to connect to the Discord API and access servers, channels, users and so on through a client that is either REST or Socket based.

Listing 21: Implementation of the DiscordRestEntityService

```
public class DiscordRestEntityService : DiscordBaseEntityService,
    IDiscordEntityService {

  public DiscordRestEntityService(ILoggerFactory loggerFactory,
      IAsyncStreamHandler streamHandler) : base(loggerFactory,
      streamHandler) {
    // Create a new client, this class is provided by the Discord.Net
    Client = new DiscordRestClient(new DiscordRestConfig() {
      DefaultRetryMode = RetryMode.AlwaysRetry
    });
  }

  public async Task Connect(string token) {
```

```
11      ((DiscordRestClient)Client).LoggedIn += Client_Ready;
12      // The login is handled in a seperate thread according to Discord.
            Net's documentation, so the method returns before the login
            has been completed. Because of this we need to add an event
            handler to the LoggedIn property.
13      await ((DiscordRestClient)Client).LoginAsync(TokenType.Bot, token)
            ;
14
15      // This is a simple workaround to wait until the connection is
            build.
16      // This is required or else the client will not be ready after the
             Connect method has been called.
17      Stopwatch sw = Stopwatch.StartNew();
18      while (!Ready && sw.ElapsedMilliseconds <= Timeout) { }
19      sw.Stop();
20    }
21
22    // Set ready to true to break the while loop in Connect
23    protected Task Client_Ready() {
24      Ready = true;
25      return Task.CompletedTask;
26    }
27
28    public async Task<DiscordServerCollection> LoadEntities() {
29      // use the client to load all accessible servers and its data
30      IEnumerable<DiscordServer> servers = await Task.WhenAll((await ((
            DiscordRestClient)Client).GetGuildsAsync()) // GetGuildsAsync
            loads all accesible servers asynchronously
31      .Select(async e => new DiscordServer() {
32        Id = e.Id,
33        Name = e.Name,
34        ParentId = null,
35        Type = DiscordEntityType.Server,
36        UserCollection = await GetUsers(e), // Map users found in data
              to custom model
37        ChannelCollection = await GetChannels(e), // Map channels found
              in data to custom model
38        RoleCollection = await GetRoles(e) // map roles found in data to
               custom model
39      }));
40
41      return new DiscordServerCollection(servers);
42    }
43  }
```

Listing 21 shows a snippet of the REST-based *IDiscordEntityService* where a new *DiscordRestClient* provided by *Discord.Net* is created and used to access all servers asynchronously (*GetGuildsAsync()*). The data is then mapped to custom models (*DiscordServer*, *DiscordUser*, ...) that contain only relevant data for the completion items. The generated data is saved to a file inside the *AppData/Local* folder.

Listing 22: Generated data


```
{
  "1157751183184760953": {
    "Type": 0,
    "UserCollection": {
      "270995861826306049": {
        "Type": 1,
        "Id": 270995861826306049,
        "Name": ".handsomebanana",
        "ParentId": 1157751183184760953
      },
      "948672754272583721": {
        "Type": 1,
        "Id": 948672754272583721,
        "Name": "Main Banana Man",
        "ParentId": 1157751183184760953
      }
    },
    "RoleCollection": {
      "1157751183184760953": {
        "Type": 2,
        "Id": 1157751183184760953,
        "Name": "@everyone",
        "ParentId": 1157751183184760953
      }
    },
    "ChannelCollection": {
      "1157751183822311494": {
        "Type": 3,
        "ChannelType": 2,
        "Id": 1157751183822311494,
        "Name": "Textkanaele",
        "ParentId": 1157751183184760953
      },
      "1157751183822311495": {
        "Type": 3,
        "ChannelType": 2,
        "Id": 1157751183822311495,
        "Name": "Sprachkanaele",
        "ParentId": 1157751183184760953
      }
    },
    "Id": 1157751183184760953,
    "Name": "Discord.NET Support Extension Testserver",
    "ParentId": null
  }
}
```


Listing 22 shows what the generated data looks like. As seen in this example the bot used to generate this data is called *Main Banana Man* which is also part of the user collection.

Load Server Collection

The data structure to hold the data in memory is based on a dictionary that has a project as key and contains the data as value. Executing this command will put the generated data for the project into the dictionary. On startup the currently selected or active project is used if the required data exists. For every other project, the *Load Server Collection* command comes in handy, to dynamically load the required data used for completion items. This is also useful if the user develops several Discord bots at the same time, because only the required data for code completion will be provided while coding.

6.3 MEF Components Project

This project contains the MEF components, which utilize functionality from the Async Completion Api mentioned in Subsection 5.3.1. These components are responsible for taking the generated data, explained in Subsection 6.2, and providing it as completion items for the Editor API.

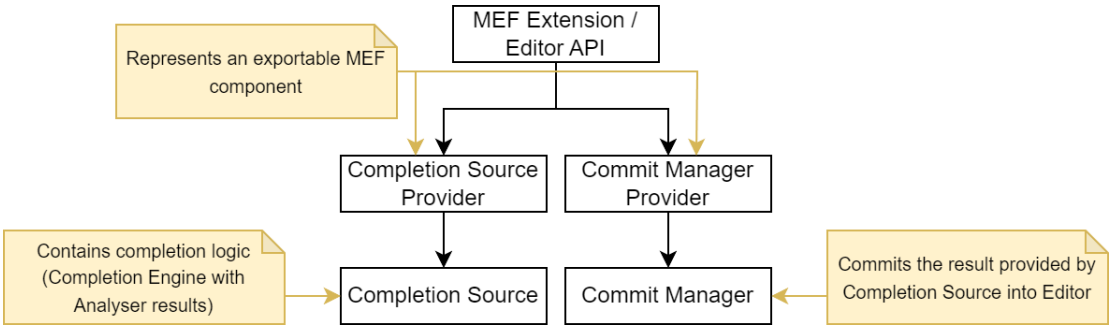

Figure 16: Workflow for the MEF Project

Figure 16 shows the basic structure, the *MEF Extension* resembling the project containing the MEF components, which include the *IAsyncCompletionSourceProvider* and *IAsyncCompletionCommitManagerProvider*.

Completion Source

Listing 23: Discord Completion Source Provider

```
1 [Export(typeof(IAsyncCompletionSourceProvider))]
2 [ContentType("CSharp")]
3 [Name("Discord Source Provider")]
4 public class AsyncDiscordCompletionSourceProvider :
      IAsyncCompletionSourceProvider {
5   public IAsyncCompletionSource GetOrCreate(ITextView textView) {
6     return new AsyncDiscordCompletionSource();
7   }
```

Listing 23 shows that classes inheriting from *IAsyncCompletionSourceProvider* implement the method *GetOrCreate(ITextView textView)*, that returns an *IAsyncCompletionSource*. That return value contains the added completion items that are pushed into the editor (which is done by Visual Studio). "When the user interacts with Visual Studio, e.g. by typing, the editor will see if it is appropriate to begin a completion session" (Wieczorek, 2018a).

Listing 24: Discord Completion Source

```
1  public class AsyncDiscordCompletionSource : IAsyncCompletionSource {
2    public async Task<CompletionContext> GetCompletionContextAsync(
         IAsyncCompletionSession session, CompletionTrigger trigger,
         SnapshotPoint triggerLocation, SnapshotSpan applicableToSpan,
         CancellationToken token) {
3      ...
4
5      // This engine is a custom implementation that performs code
           analysis and is part of a DI container from which the instance
            is resolved.
6      IDiscordCompletionEngine engine = supportPackageContainer.Resolve<
           IDiscordCompletionEngine>();
7
8      try {
9        ...
10       // The documentIdentifier is set in InitializeCompletion(..)
11       Microsoft.CodeAnalysis.Document document = vsWorkspace.
             CurrentSolution.GetDocument(documentIdentifier);
12
13       ...
14
15       SemanticModel semanticModel = await document.
             GetSemanticModelAsync(token);
16       SyntaxToken triggerToken = (await document.GetSyntaxRootAsync(
             token)).FindToken(triggerLocation);
17
18       // The engine starts the required code analysers that will build
              completions based on the analysed code
19       DiscordCompletionItem[] completions = await engine.
             ProcessCompletionAsync(vsWorkspace.CurrentSolution,
             semanticModel, triggerToken);
20
21       ...
22       // The CompletionContext takes an ImmutableArray of
             CompletionItem, so the just now created
             DiscordCompletionItem[] is mapped to a CompletionItem[].
23       return new CompletionContext(completions.Select(e => {
24         CompletionItem ci = new CompletionItem(/* Mapping of
               completion items*/);
25         ci.Properties.AddProperty("description", e.Description); //
               The discription called in GetDescriptionAsync(..)
26         return ci;
27       }).ToImmutableArray());
28     }
29     catch (Exception ex) {
30       ...
```

```
31      }
32
33      return default;
34    }
35
36    public async Task<object> GetDescriptionAsync(
          IAsyncCompletionSession session, CompletionItem item,
          CancellationToken token) {
37      // The discription of an item displayed in the IntelliSense
            completion list
38    }
39
40    public CompletionStartData InitializeCompletion(CompletionTrigger
          trigger, SnapshotPoint triggerLocation, CancellationToken token)
           {
41      // Initialization work i.e. setting the current document that is
            analysed in GetCompletionContextAsync(..)
42      // Also defines if this completion source (this class)
            participates in completion (which means GetCompletionContext
            (..) will be called)
43    }
44  }
```

In Listing 23 a new *AsyncDiscordCompletionSource* is created. The API will then proceed to call the methods defined in the *IAsyncCompletionSource* interface if IntelliSense is triggered. "We will attempt to get completion items by asynchronously calling GetCompletionContextAsync where you provide completion items" (Wieczorek, 2018a). As shown in Listing 24, the methods inside *AsyncDiscordCompletionSource*, *GetCompletionContextAsync*, *GetDescriptionAsync* and *InitializeCompletion* are implemented based on this use case and are explained each as comment in the Listing.

Completion Commit Manager

Listing 25: Discord Completion Commit Manager Provider

```
1  [Export(typeof(IAsyncCompletionCommitManagerProvider))]
2  [ContentType("CSharp")]
3  [TextViewRole(PredefinedTextViewRoles.Editable)]
4  [Order(Before = "default")]
5  [Name("Discord Commit Provider")]
6  public class AsyncDiscordCompletionCommitManagerProvider :
       IAsyncCompletionCommitManagerProvider {
7    public IAsyncCompletionCommitManager GetOrCreate(ITextView textView)
         {
8      return new AsyncDiscordCompletionCommitManager();
9    }
10 }
```

Similar as in Listing 23, Listing 25 shows that classes inheriting from *IAsyncCompletionCommitManagerProvider* implement the method *GetOrCreate(ITextView textView)*, that returns an *IAsyncCompletionCommitManager*. That return value contains the logic responsible for committing completion items into the editor. "We use this interface to determine under which circumstances to commit (insert the completion text into the text buffer and close the completion UI) a completion item." (Wieczorek, 2018a)

Listing 26: Discord Completion Commit Manager

```
1  public class AsyncDiscordCompletionCommitManager :
       IAsyncCompletionCommitManager {
2    public IEnumerable<char> PotentialCommitCharacters => ".".
       ToCharArray();
3
4    public bool ShouldCommitCompletion(IAsyncCompletionSession session,
       SnapshotPoint location, char typedChar, CancellationToken token)
        {
5      return false; // No custom commit characters should commit the
         completion item into the editor
6    }
7
8    public CommitResult TryCommit(IAsyncCompletionSession session,
       ITextBuffer buffer, CompletionItem item, char typedChar,
       CancellationToken token) {
9      // If the item type matches, insert the currently selected
         completion item with all required information into the
         textbuffer
10     if (item.Source is AsyncDiscordCompletionSource) {
11       buffer.Replace(session.ApplicableToSpan.GetSpan(buffer.
           CurrentSnapshot), item.InsertText + $" /* {item.DisplayText}
            ({item.Suffix}) */");
12       return CommitResult.Handled;
13     }
14
15     return CommitResult.Unhandled;
16   }
17 }
```

In Listing 25 a new *AsyncDiscordCompletionCommitManager* is created. This class is responsible for managing how the completion items added by the *AsyncDiscordCompletionSource* (from Listing 24) are committed into the editor. "When we first create the completion session, we access the PotentialCommitCharacters property that returns characters that potentially commit completion when user types them. We access this property, therefore it should return a preallocated array. Typically, the commit characters include space and other token delimeters such as ., (,)." (Wieczorek, 2018b)
It is also stated, that tab and enter are handled seperately. So if a character is a commit character, it needs to be added to the mentioned *PotentialCommitCharacters*. (Wieczorek, 2018b)

6.4 Code Analyser

In Listing 24 the current document is available through working with the Development Tools Environment (DTE). The DTE is used to access many different kinds of features within Visual Studio. Therefore the currently open document id is accessible, which is then utilized to access the current document. This information is crucial to get the semantic model, which is also mandatory for analysing the users code. The method *GetCompletionContextAsync* from Listing 24 provides a *SnapshotPoint* as parameter. With a *SnapshotPoint* and the available document, the *SyntaxToken*, where IntelliSense has been triggered, can be acquired.

Listing 27: Getting the semantic model and syntax token

```
1 SemanticModel semanticModel = await document.GetSemanticModelAsync(
      token);
2 SyntaxToken triggerToken = (await document.GetSyntaxRootAsync(token)).
      FindToken(triggerLocation);
```

Listing 27 shows the above mentioned logic. The *document* represents the file that is currently open in the editor and the *triggerLocation* contains information about where in the currently open file IntelliSense was triggered. This is always where the cursor position. The *token* is a *CancellationToken*, which is a good practice to use when writing asynchronous code. The semantic model and syntax token are then used to get the necessary information about the users current state of code.

Workflow

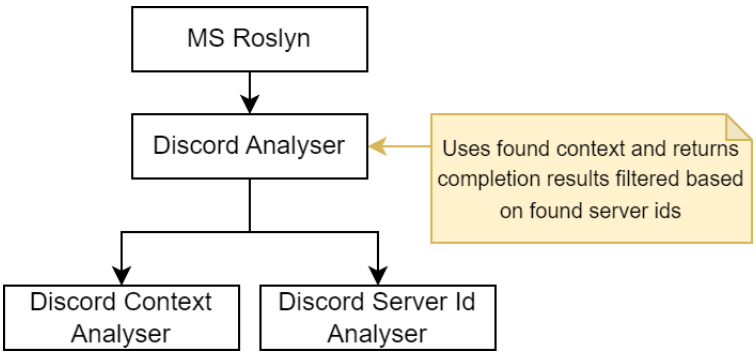

Figure 17: Workflow for the implemented code analyser using Roslyn

Figure 17 displays a structure of 3 analysers, the *Discord Analyser* that collects information through the sub analysers *Discord Context Analyser* and *Discord Server Id Analyser*. The *Discord Analyser* is called by the *IDiscordCompletionEngine*. The use of which is shown at Listing 24.

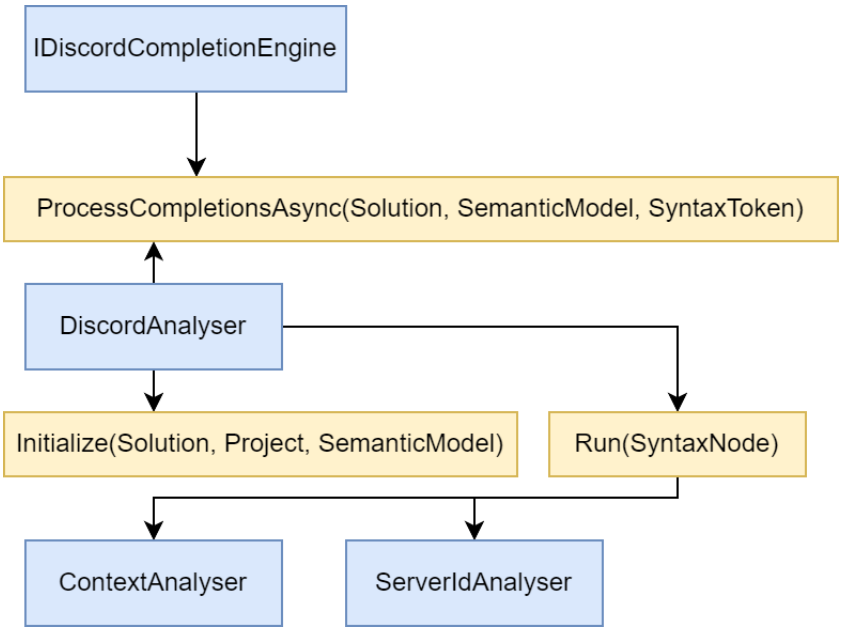

Figure 18: Analysis worklow with the IDiscordCompletionEngine as starting point

Figure 18 shows the analysis process that is started every time the IntelliSense is triggered. In this scenario, the *IDiscordCompletionEngine* calls *ProcessCompletionsAsync*. Since the generated data is stored per project using a dictionary, the current project is required and used to read the generated data for this project. It also creates the *DiscordAnalyser* that utilizes the *ContextAnalyser* and the *ServerIdAnalyser*. The *ContextAnalyser* tries to evaluate the context, which consists of *Server*, *User*, *Role* and *Channel*. If the context is *Server* or no context is found, the *ServerIdAnalyser* is not called, hence there is no need to specify the server for appropriate filtering. The *ServerIdAnalyser* tries to find a server id from the *triggerLocation* as starting point, which is then used to filter the generated data. Since Roslyn maintains the compiler API (Figure 9), there is access to a big variety of classes that represent different types of code.

Syntax Showcase

Listing 28: Code snippet from the context analyser

```
1 ...
2 private async Task ResolveNode(SyntaxNode node) {
3   switch (node) {
4     case InvocationExpressionSyntax invocation:
5     CheckForContext(invocation);
6     break;
7     case BinaryExpressionSyntax binary when binary.Kind() ==
          SyntaxKind.EqualsExpression || binary.Kind() == SyntaxKind.
          NotEqualsExpression:
8     await ResolveNode(binary.Left);
```

```
9       break;
10      case MemberAccessExpressionSyntax memberAccess:
11      await ResolveNode(memberAccess.Expression);
12      break;
13      case ConditionalAccessExpressionSyntax conditionalAccess:
14      await ResolveNode(conditionalAccess.Expression);
15      break;
16      case IdentifierNameSyntax identifier:
17      CheckForContext(identifier);
18      break;
19      case SwitchStatementSyntax switchStatement:
20      await ResolveNode(switchStatement.Expression);
21      break;
22      case ArgumentSyntax argument:
23      await ResolveArgument(argument);
24      break;
25      case ArgumentListSyntax argumentList:
26      await ResolveNode(argumentList.Parent);
27      break;
28    }
29  }
```

For instance when taking a look at Listing 28, there are many different classes used that represent a C# *SyntaxNode*, like an *InvocationExpressionSyntax* in line 4, which stands for a method call. When utilizing these things, the users code can be queried efficiently. But it is again important to state, that this analysis process is called each time IntelliSense will trigger. To not negatively impact the users coding experience with bad performance, it is important that the algorithms analysing code are fast.

Example of Context Detection

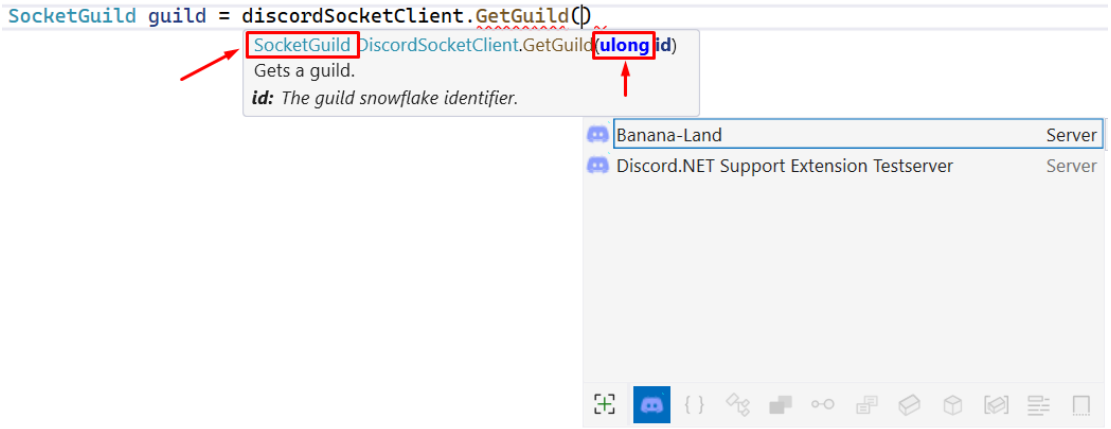

Figure 19: Example of how the current context can be determined

Figure 19 shows a code snippet where a server is retrieved using a *DiscordSocketClient* (this is the *discordSocketClient* variable's type). The goal is to find out whether the cursor is at a position where an id can be provided, and if so, what kind of id is required (*Server, User, ..*). When IntelliSense triggers between the *GetGuild()* method parentheses, so where the cursor is positioned, the *DiscordContextAnalyser* starts analysing. First the type of the parameter is checked. Discord uses so called Snowflake IDs for their entities, which represent an unsigned long in C# . As seen in the Figure, the type matches. After the parameter type is confirmed, the methods return type is checked and if that also matches, the completion items are prepared and provided as suggestions. This is only a simple scenario, code can become more complex which can complicate the analysis, like for example when using LINQ.

Example of Server Id Resolving

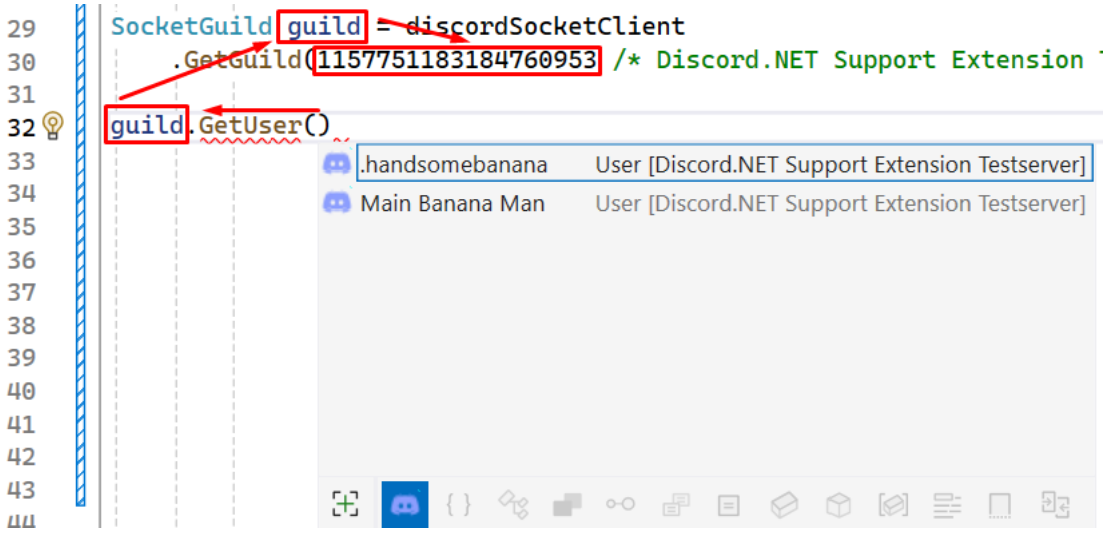

Figure 20: Example of how the correct server id is resolved

Figure 20 shows how a server id is successfully resolved. If the context detection is successful, which in this case is *User*, the *DiscordServerIdAnalyser* starts analysing. The context detection provides the *SyntaxNode* the context was detected on, but only if a context was found. In this Figure the determined *SyntaxNode* is *GetUser()* which represents an *InvocationExpressionSyntax*. Then the variable *guild*, where that method is called on, is checked. The variable *guild* in Line 32 is of type *IdentifierNameSyntax*, that contains the declaring syntax reference which is used to get the variable declaration in Line 29. This *SyntaxNode* is of type *VariableDeclaratorSyntax* and contains a property representing the initializer. So now the initializer is checked and from there on the server id is successfully resolved by checking the *GetGuild* methods return type and its parameter. This is again a very simple example, since code can become more complex quickly, which can also complicate the analysis. For this reason it is important to always check the termination conditions before jumping right into development. Also it is recommended to get familiar with syntax and semantics using the Syntax Visualizer introduced in Subsection 5.2.3.

7 Results

To demonstrate the full results process, the following section also presents the exact way in which these results were achieved. That includes the creation of a new project and the data generation for the Discord bot named *Main Banana Man*.

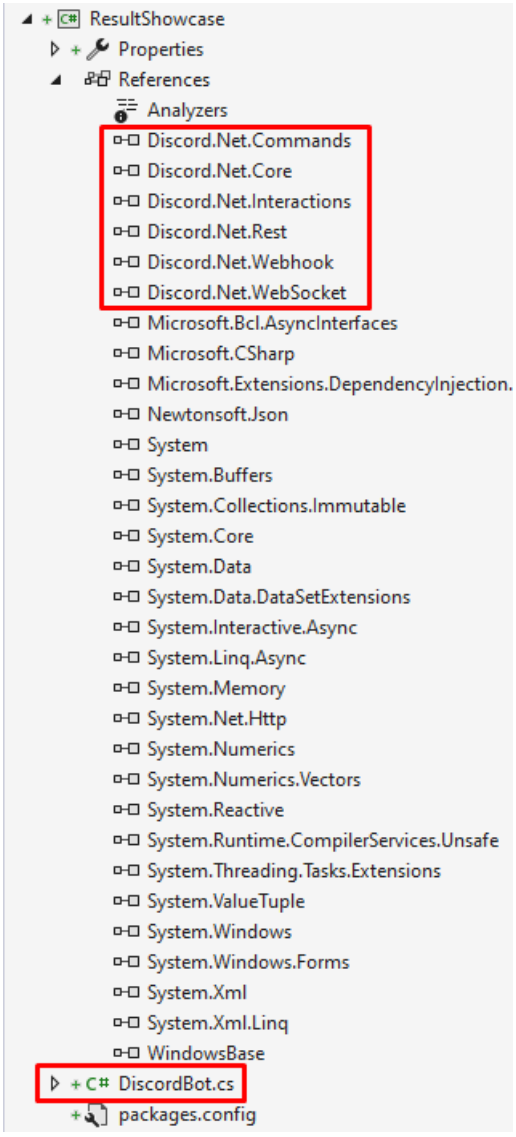

Figure 21: New project for showcase

Figure 21 represents the new project containing a *DiscordBot.cs* file and the installed *Discord.Net* NuGet packages.

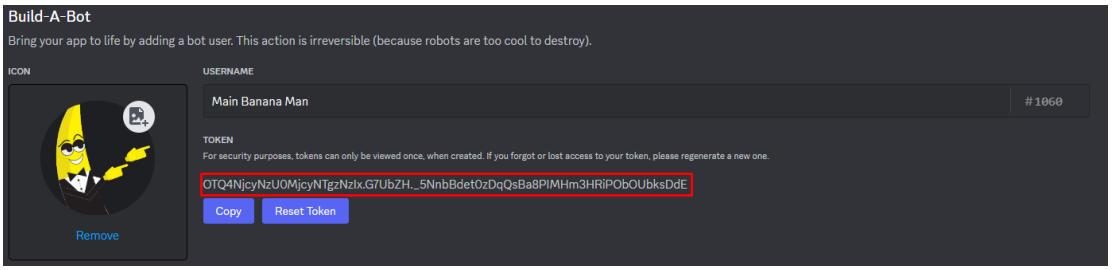

Figure 22: The created bot with its token

Figure 22 shows the bot that was created on `https://discord.com/developers/applications/`. The process of creating a bot in the API is not explained in detail, but can be found on various websites. YouTube and many others offer a large amount of tutorials for this. Just to name one, a good explanation can be found on `https://www.ionos.com/digitalguide/server/know-how/creating-discord-bot/`. The created bot contains a token that is required to build a connection to the Discord API. This token can also be seen in Figure 22. It is also important that this token is not exposed to the public, because anybody will be able to connect to the API using this bot. If the token is exposed either way, it can be reset with the *Reset Token* button as seen in Figure 22.

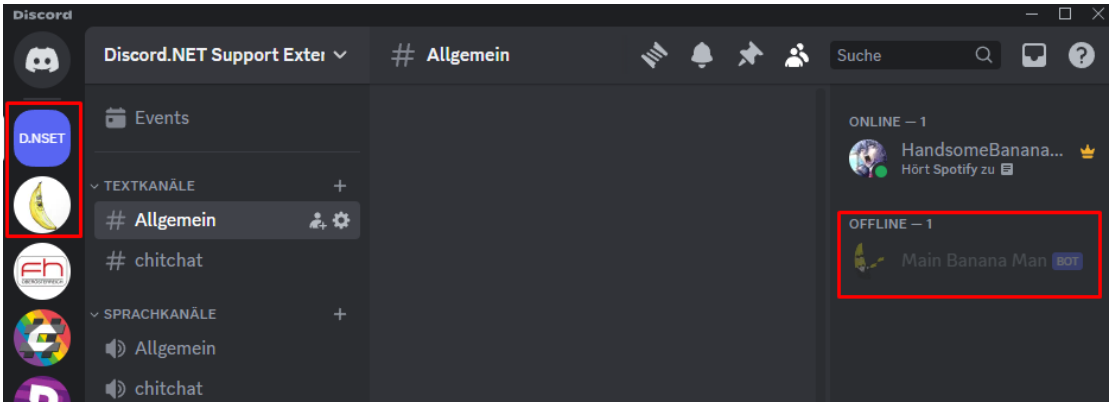

Figure 23: Small overview of the server

Figure 23 shows the Discord desktop application. On the right side the user list is displayed. As can be seen, the bot *Main Banana Man* is part of the server. On the left side the server list, the current logged in user is part of, can be seen. The *Main Banana Man* bot is part of the 2 servers at the top, which are also highlighted in Figure 23. The first one is called *Discord.NET Support Extension Testserver* and the second one is called *Banana Land*. This means that the data for these 2 servers can be generated, provided that the bot has the necessary read permissions.

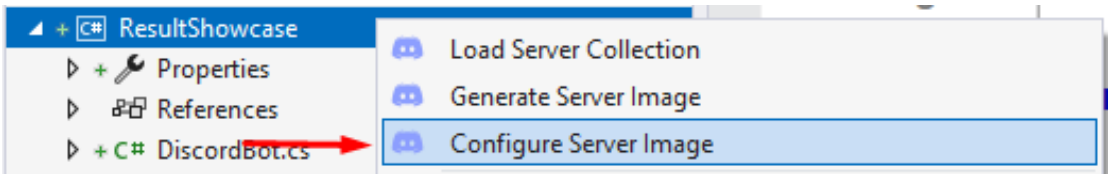

Figure 24: Highlighted Server Image Configuration command

As seen in Figure 24 the next step is to configure the data generation.

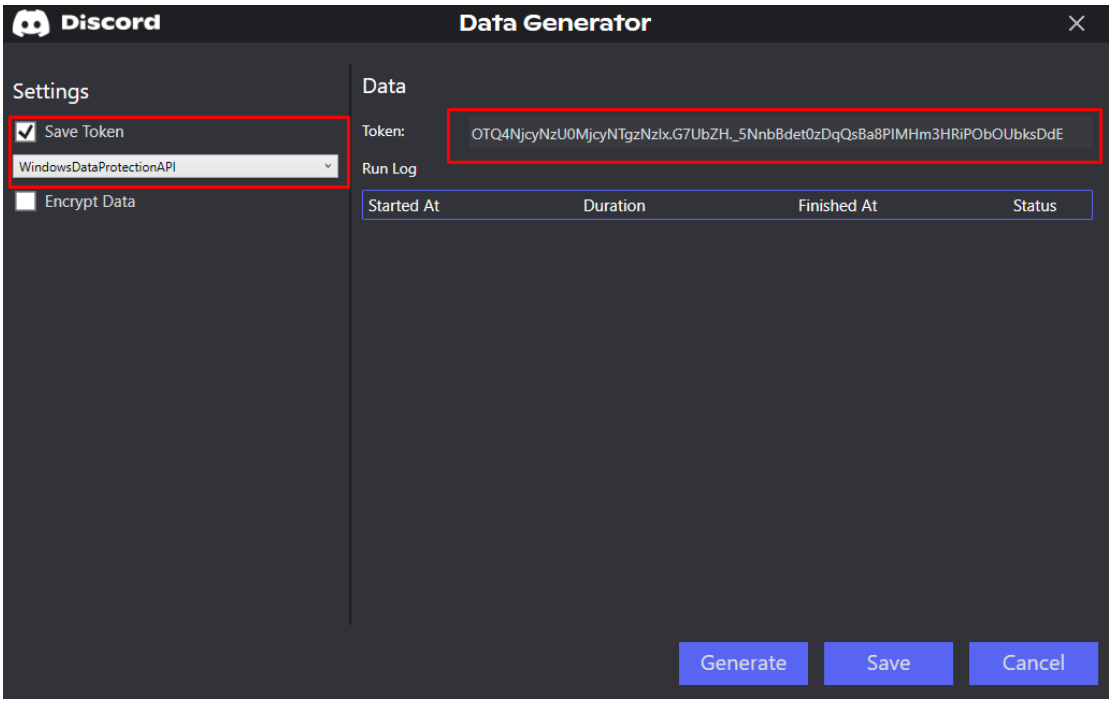

Figure 25: Configured Data Generation

Looking Figure 25, the configuration is done. It can be now saved using the *Save* button for later data generation, or the data can directly be generated using the *Generate* button. The configuration is saved either way. So the next step is to actually generate the data.

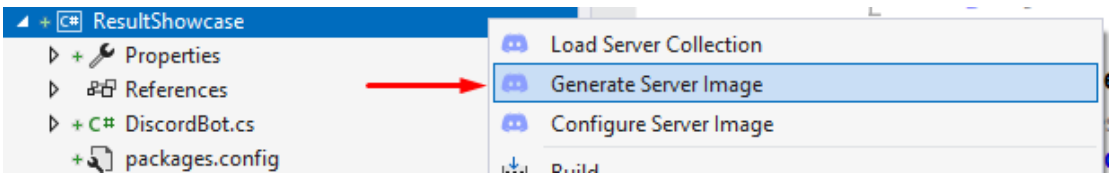

Figure 26: Highlighted Server Image Generation command

To generate the data, the project is right clicked and the command *Generate Server Image* is executed.

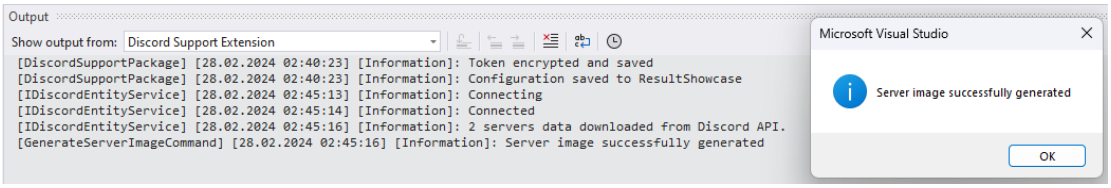

Figure 27: Log for data generation

Figure 27 shows the output log after the *Generate Server Image* command was executed. Also a message box is shown containing the information about the executions result state. When generating new data, it is automatically loaded so there is no need to explicitly call the *Load Server Collection* command for this project. But this is only the case when Visual Studio is not closed. When reopening Visual Studio, executing this command is required. Now that the data was successfully generated, it is used to provide code completion.

Server completion items

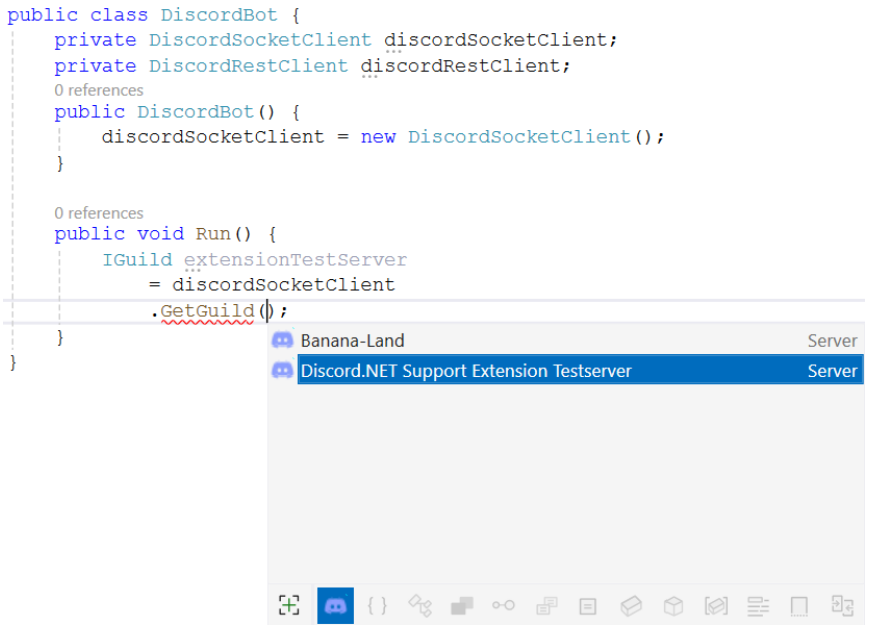

Figure 28: Completion items for getting a server

Figure 28 displays the possible completion items for this context. In this case, the found context is *Server*, so each server found in the generated data (Paragraph 6.2) is going to be provided as completion item. When looking back at Figure 23, the 2 suggestions are based on the servers the bot is part of. This action's responsible component *IAsyncCompletionSource* is described in Listing 24.

```
public class DiscordBot {
    private DiscordSocketClient discordSocketClient;
    private DiscordRestClient discordRestClient;
    0 references
    public DiscordBot() {
        discordSocketClient = new DiscordSocketClient();
    }

    0 references
    public void Run() {
        IGuild extensionTestServer
            = discordSocketClient
            .GetGuild(1157751183184760953 /* Discord.NET Support Extension Testserver (Server) */);
    }
}
```

Figure 29: Committed server completion item

Figure 29 shows how the selected completion item is committed into the editor. This action's responsible component *IAsyncCompletionCommitManager* is described in Listing 26.

User completion items

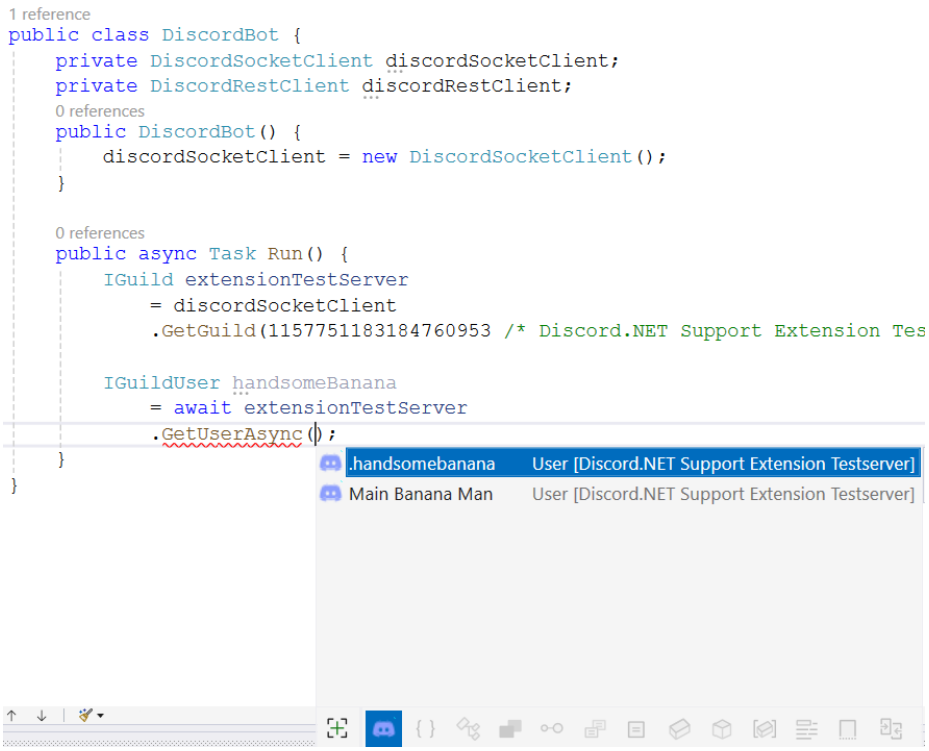

Figure 30: Completion items for getting a user that is part of the given server

It is important to note, that when the given context is not *Server*, like in this case *User*, not every user found in the generated data is taken and provided as completion item. The *Server Id Analyser* mentioned in Subsection 6.4 determines on which server-variable the *GetUserAsync* method is called. So for this example, when requesting the user completion items, the algorithm knows that this method is called on the *extensionTestServer* variable and will proceed to get the id connected with this variable, which is the static value inside the *GetGuild* method. If the algorithm is unable to find a server id, it will provide all users found in the generated data. But as seen in Figure 30, the users are filtered correctly, since Figure 23 shows that these 2 users are part of the server.

```
1 reference
public class DiscordBot {
    private DiscordSocketClient discordSocketClient;
    private DiscordRestClient discordRestClient;
    0 references
    public DiscordBot() {
        discordSocketClient = new DiscordSocketClient();
    }

    0 references
    public async Task Run() {
        IGuild extensionTestServer
            = discordSocketClient
            .GetGuild(1157751183184760953 /* Discord.NET Support Extension Testserver (Server) */);

        IGuildUser handsomeBanana
            = await extensionTestServer
            .GetUserAsync(270995861826306049 /* .handsomebanana (User [Discord.NET Support Extension Testserver]) */);
    }
}
```

Figure 31: Committed user completion item

Figure 31 shows how the selected completion item is committed into the editor.

Channel completion items

When again taking a look at the Discord Server (Figure 23), there are 6 channels visible. 2 text channels *Allgemein* and *chitchat*, 2 voice channels *Allgemein* and *chitchat*, and the 2 category channels that contain the voice and text channels *Textkanaele* and *Sprachkanaele*. Since *Channel* is a supported context, these entities will also be suggested as completion item.

```
public class DiscordBot {
    private DiscordSocketClient discordSocketClient;
    private DiscordRestClient discordRestClient;
    0 references
    public DiscordBot() {
        discordSocketClient = new DiscordSocketClient();
    }

    0 references
    public async Task Run() {
        SocketGuild extensionTestServer
            = discordSocketClient
            .GetGuild(1157751183184760953 /* Discord.NET Support Extension Test

        ITextChannel textChannel = extensionTestServer
            .GetTextChannel()
    }
}
```

```
Allgemein    Text [Discord.NET Support Extension Testserver]
chitchat     Text [Discord.NET Support Extension Testserver]
```

Figure 32: Completion items for getting a text channel that is part of the given server

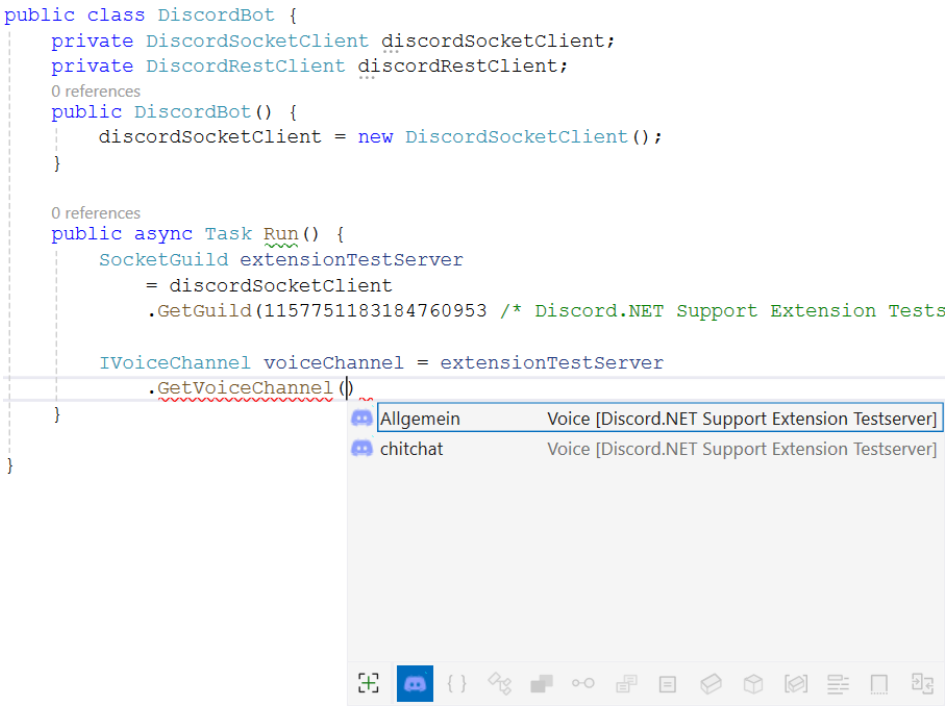

Figure 33: Completion items for getting a voice channel that is part of the given server

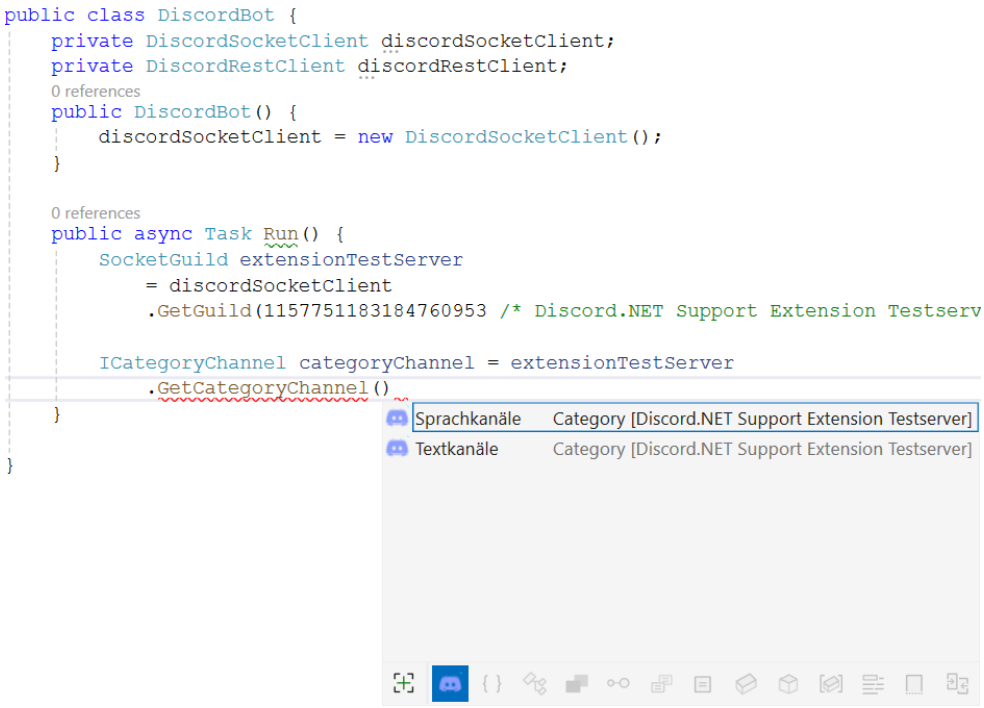

Figure 34: Completion items for getting a category channel that is part of the given server

Figure 32, 33 and 34 show the suggestions for text, voice and category channels. As mentioned, when looking at the Discord server (Figure 23), there are the 6 channels existent.

Completion items when using LINQ

More complex coding scenarios are also supported, like when using LINQ.

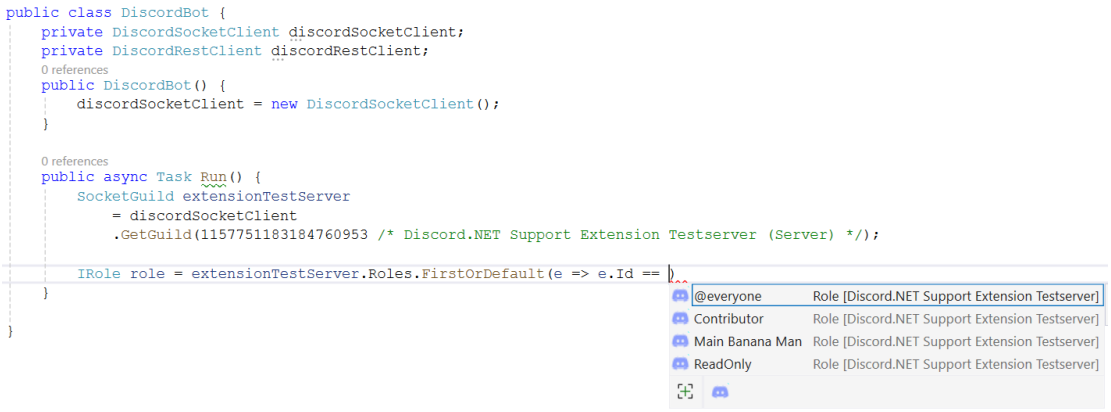

Figure 35: Role suggestions in LINQ method

As shown in Figure 35, when using Linq methods the context is determined and the suggestions are provided correctly.

8 Closing

This thesis concludes with the development of a Visual Studio Extension aimed at improving the process of creating Discord bots. Through the integration of Visual Studio Extensibility and Microsoft Roslyn, this project has provided tools that enhance the coding experience by offering intelligent code completion. This improvement made the bot development process more accessible and efficient. Reflecting on the objectives described at the start, this work has met its goals. The extension has become a valuable tool for developers, enabling them to build bots quicker than before. While the results are fulfilled, there is potential for further development. Future improvements could include adding more API integrations, improving the user interface for a better experience and a more complex and profound code analysis to cover more development challenges. Overall, this project has made a meaningful contribution to the field of Discord bot development and serves as a foundation for future research and development in this area.

References

Agnihotri, Advait, Harshit Gautam, and Hari Singh (2022). "Discord Bot". In.

Developer.com (2012). *An Introduction to Microsoft Roslyn*. https://www.developer.com/microsoft/an-introduction-to-microsoft-roslyn/ (visited on 01/07/2024).

Discord (2022). *What is Discord?* https://discord.com/safety/360044149331-what-is-discord (visited on 07/08/2023).

Discord.Net (2015). *Discord.Net*. https://discordnet.dev/ (visited on 11/23/2023).

GameInfoHub (2023). *Inside the Discord Revolution: Insights into Gaming's Powerful Communication Platform*. https://gameinfohub.com/inside-the-discord-revolution-insights-into-gamings-powerful-communication-platform/ (visited on 01/05/2024).

Microsoft (2021a). https://github.com/Microsoft/vs-editor-api (visited on 01/07/2024).

— (2021b). *Understand the .NET Compiler Platform SDK model*. https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/compiler-api-model (visited on 01/07/2024).

— (2021c). *Visual Studio Extensibility Cookbook*. https://www.vsixcookbook.com/ (visited on 01/07/2024).

— (2021d). *Work with a workspace*. https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/work-with-workspace (visited on 01/07/2024).

— (2021e). *Work with semantics*. https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/work-with-semantics (visited on 01/07/2024).

— (2021f). *Work with syntax*. https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/work-with-syntax (visited on 01/07/2024).

— (2022). *Explore code with the Roslyn syntax visualizer in Visual Studio*. https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/syntax-visualizer?tabs=csharp (visited on 01/07/2024).

— (2023). *Visual Studio Extension*. https://learn.microsoft.com/en-us/visualstudio/extensibility/starting-to-develop-visual-studio-extensions?view=vs-2022 (visited on 11/23/2023).

TheEnlightenedMindset (2023). *Who Invented Discord? The Story of Its Innovators and Developers*. https://www.tffn.net/who-invented-discord/ (visited on 01/05/2024).

ToptalDevelopers (2022). *How to Make a Discord Bot: an Overview and Tutorial*. https://www.toptal.com/chatbot/how-to-make-a-discord-bot (visited on 01/05/2024).

Vasani, Manish (2017). *Roslyn Cookbook*. Packt Publishing Ltd.

Wieczorek, Amadeusz (2018a). *Async Completion API discussion*. `https://github.com/microsoft/vs-editor-api/issues/9` (visited on 09/10/2023).

— (2018b). *Modern completion API*. `https://github.com/Microsoft/vs-editor-api/wiki/Modern-completion-API` (visited on 09/10/2023).